

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
PUDUCHERRY TECHNOLOGICAL UNIVERSITY
PUDUCHERRY – 605 014.**

BONAFIDE CERTIFICATE

This is to certify that the Project work titled “**HYBRID MULTI-LANGUAGE CODE AWARE SEARCH ENGINE**” is a bonafide work done by **S Sai Madhav (2201107103)**, **Siripurapu Khusheel Manideep (2201107096)**, **P Jaswanth Naidu (2201107074)**, **S Charan Teja (2201107100)** in partial fulfillment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of the **Puducherry Technological University** and that this work has not been submitted for the award of any other degree of this/any other institution.

Project Guide
(Dr. P. Salini)

Head of the Department
(Dr. E. ILAVARASAN)

Submitted for the University Examination held on _____

Project Coordinator

External Examiner

ACKNOWLEDGEMENT

I am deeply indebted to **Dr. P. Salini**, Professor Department of Computer Science and Engineering, Puducherry Technological University, Puducherry, for his valuable guidance throughout this project.

I also express my heart-felt gratitude to **Dr. E. Ilavarasan**, Professor & Head (CSE) for giving constant motivation in succeeding my goal.

With profoundness I would like to express my sincere thanks to **Dr. S. Mohan**, The Vice-Chancellor, Puducherry Technological University, for his kindness in extending the infrastructural facilities to carry out my project work successfully.

I would be failing in my duty if I do not acknowledge the efforts of the Project Co-ordinator, **Dr. M. Thirumaran**, and the Project Review Panel members, viz. **Dr. R. Manoharan, Dr. S. Lakshmana Pandian, Dr. R. Sarala, Dr. N. Sivakumar** for shaping our ideas and constructive criticisms during project review.

I also express my thanks to all the **Faculty** and **Technical Staff** members of the CSE department for their timely help and the **Central Library** for facilitating useful reference materials.

I would be failing in my duty if I don't acknowledge the immense help extended by my friends, who have always been with me in all my trials and tribulations and encouraging me to complete.

S Sai Madhav
S Khusheel Manideep
P Jaswanth Naidu
S Charan Teja

ABSTRACT

Code search is a fundamental requirement in modern software development, enabling developers to efficiently locate relevant code snippets, understand codebases, and promote software reuse. Traditional code search systems rely on keyword-based information retrieval techniques, treating code as plain text and ignoring its structural and semantic properties. This often leads to inaccurate results and poor alignment with user intent. Although deep learning-based approaches improve semantic understanding, they introduce challenges such as high computational cost, dependence on large datasets, and lack of explainability.

To address these issues, this project proposes a “Hybrid Multi-Language Code-Aware Search Engine with Explainable Ranking”. The system integrates structural and semantic techniques to improve retrieval accuracy while maintaining efficiency and interpretability. It supports multiple programming languages and processes real-world repositories by extracting structural elements such as functions, classes, and methods, which are further divided into fine-grained snippets for precise indexing and retrieval.

The system employs a hybrid ranking approach that combines structural matching and semantic similarity. Structural ranking ensures fast and interpretable results using features like identifiers and keyword overlap, while semantic similarity captures the intent behind user queries. An explainable ranking mechanism is incorporated to provide transparency by showing how different factors contribute to the final results.

The system is evaluated using standard metrics such as Precision@K, Recall@K, Mean Reciprocal Rank (MRR), NDCG, Hit Rate, and Latency. Results demonstrate improved accuracy and efficiency compared to traditional methods, without the overhead of deep learning models.

Overall, the proposed system offers a scalable, efficient, and interpretable solution for intelligent code search, enhancing developer productivity and supporting effective code reuse.

TABLE OF CONTENTS

CHAP NO.	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	i
	ACKNOWLEDGEMENT	ii
	ABSTRACT	iii
	LIST OF FIGURES	iv
	LIST OF TABLES	vi
	LIST OF ABBREVIATIONS	vii
	LIST OF SYMBOLS	viii
1	INTRODUCTION	1
	1.1 Background of Code Search	1
	1.2 Evolution of Code Search Systems	2
	1.3 Problem Statement	3
	1.4 Motivation	4
	1.5 Objectives of the Project	5
	1.6 Scope of the Project	6
	1.7 Key Contributions of the Work	7
	1.8 Organization of the Report	8
2	LITERATURE REVIEW & RESEARCH ANALYSIS	11
	2.1 Overview of Code Search Techniques	11
	2.2 Traditional Information Retrieval Methods	11
	2.3 Structural Code Search Approaches (AST-Based)	12
	2.4 Deep Learning-Based Code Search Models	13
	2.5 Cross-Domain Code Search (Base Paper Analysis)	14

2.6	Semantic Gap and OOV Problem	15
2.7	Comparative Study of Existing Systems	15
2.8	Research Gaps Identified	16
3	EXISTING SYSTEM ANALYSIS	21
3.1	Overview of Existing Systems	21
3.2	Architecture of Traditional Code Search Systems	22
3.3	Modules of Existing System	23
3.3.1	Code Repository Module	23
3.3.2	Text-Based Preprocessing	24
3.3.3	Flat Indexing	24
3.3.4	Query Processing	24
3.3.5	Similarity Matching & Ranking	24
3.3.6	Result Presentation	24
3.4	Limitations of Existing Systems	24
3.5	Need for Proposed System	25
4	PROPOSED HYBRID CODE-AWARE SEARCH SYSTEM	27
4.1	Overview of Proposed System	27
4.2	Design Goals	27
4.3	System Architecture	28
4.4	System Workflow	30
4.5	Detailed Module Description	31
4.5.1	GitHub Repository Loader	32
4.5.2	Multi-Language File Loader	32
4.5.3	Language-Specific Parsing Module	33
4.5.4	Snippet Extraction Engine	33
4.5.5	Query Processing Engine	33

4.6	Hybrid Ranking System	34
4.6.1	Structural Ranking Mechanism	34
4.6.2	Semantic Similarity Mechanism	34
4.6.3	Hybrid Ranking Strategy	35
4.7	Explainable Ranking Mechanism	35
4.8	Multi-Language Support Design	36
4.9	Frontend Dashboard & UI Design	37
5	MATHEMATICAL MODEL & ALGORITHMIC DESIGN	38
5.1	Problem Formulation	38
5.2	Structural Scoring Model	38
5.3	Semantic Similarity Model	39
5.4	Hybrid Scoring Function	39
5.5	Ranking Algorithm	39
5.6	Evaluation Metrics Formulation	40
6	IMPLEMENTATION DETAILS	42
6.1	Technology Stack	42
6.2	Backend Implementation	43
6.3	Frontend Implementation	44
6.4	Code Parsing & Processing	45
6.5	Indexing Mechanism	45
6.6	Query Execution Pipeline	46
6.7	Explainability Implementation	47
7	EXPERIMENTAL SETUP & RESULTS	49
7.1	Dataset Description	49
7.2	Experimental Setup	49
7.3	Performance Metrics	50

7.4 Results and Analysis	51
7.5 Graphical Analysis	52
7.6 Comparative Analysis	56
7.7 Performance Discussion	56
8 SYSTEM EVALUATION & DISCUSSION	58
8.1 Strengths of Proposed System	58
8.2 Limitations	59
8.3 Comparison with Base Paper (ACD-SA)	59
8.4 Practical Use Cases	60
9 CONCLUSION & FUTURE WORK	61
9.1 Conclusion	61
9.2 Future Enhancements	61
9.3 Real-World Applications	62
SUSTAINABLE DEVELOPMENT GOALS	64
REFERENCES	65

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Existing System Architecture Diagram	22
Figure 3.2	Existing System Workflow Diagram	23
Figure 4.1	Proposed System Architecture Diagram	29
Figure 4.2	Module Interaction Diagram	31
Figure 4.3	Hybrid Ranking Mechanism Diagram	34
Figure 4.4	Explainable Ranking Flow Diagram	36
Figure 6.1	Backend Processing Flow	43
Figure 6.2	User Interface – Input Page	44
Figure 6.3	Query Processing Pipeline	46
Figure 6.4	Search Results with Explainability	48
Figure 7.1	Precision@K Comparison Graph	52
Figure 7.2	Recall@K Comparison Graph	53
Figure 7.3	Hit@1 Comparison Graph	53
Figure 7.4	Hit@5 Comparison Graph	54
Figure 7.5	Hit@10 Comparison Graph	54
Figure 7.6	Mean Reciprocal Rank (MRR) Comparison	55
Figure 7.7	Latency Comparison Graph	55

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
Table 2.1	Comparison Between Base Paper and Proposed System	17
Table 2.2	Literature Review of Code Search Techniques	17
Table 3.1	Modules in the Existing System	23
Table 4.1	Modules in the Proposed System	32
Table 7.1	Experimental Setup	50
Table 7.2	Evaluation Metrics Description	51
Table 7.3	Performance Results	57

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AST	Abstract Syntax Tree
IR	Information Retrieval
TF-IDF	Term Frequency–Inverse Document Frequency
BM25	Best Matching 25
NLP	Natural Language Processing
ML	Machine Learning
DL	Deep Learning
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
API	Application Programming Interface
UI	User Interface
IDE	Integrated Development Environment
OOV	Out-of-Vocabulary
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
DCG	Discounted Cumulative Gain
IDCG	Ideal Discounted Cumulative Gain
P@K	Precision at K
R@K	Recall at K
Hit@K	Hit Ratio at K
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
CPU	Central Processing Unit
GPU	Graphics Processing Unit

LIST OF SYMBOLS

SYMBOL	DESCRIPTION
Q	User query
S	Set of all code snippets
S_i	Individual code snippet
T_Q	Set of terms in query
T_{S_i}	Set of terms in snippet
$\vec{Q}$	Query vector representation
$\vec{S}_i$	Code snippet vector representation
$\text{Score}(Q, S_i)$	Final relevance score
Score_struct	Structural similarity score
Score_sem	Semantic similarity score
w_1, w_2	Weight parameters
K	Number of top results
N	Total number of queries
rank_i	Rank of first relevant result
$ \cdot $	Cardinality (size of a set)
$\cap$	Intersection of sets
$\cdot$	Dot product
$\ \cdot\ $	Vector magnitude (norm)
Σ	Summation operator
DCG	Discounted Cumulative Gain
IDCG	Ideal Discounted Cumulative Gain
Hit@K	Hit Ratio metric
Latency	Time taken to process a query

CHAPTER I

INTRODUCTION

1.1 BACKGROUND OF CODE SEARCH

In recent years, the field of software development has experienced unprecedented growth due to the rapid expansion of digital technologies and the widespread adoption of open-source platforms. Large-scale repositories such as GitHub, GitLab, and Bitbucket host millions of projects containing billions of lines of source code across diverse programming languages and domains. This vast availability of code has significantly transformed development practices, enabling programmers to reuse existing implementations, accelerate development cycles, and improve software quality. However, the abundance of code has also introduced the challenge of efficiently locating relevant code fragments within massive and heterogeneous repositories.

Code search systems have emerged as essential tools to address this challenge by enabling developers to retrieve relevant code snippets based on user queries. These queries are often expressed in natural language, reflecting the developer's intent rather than exact code syntax. For instance, a developer may search for "merge sort implementation" or "file reading in Python," expecting the system to return accurate and usable code snippets. Therefore, the effectiveness of a code search system depends on its ability to bridge the gap between natural language queries and structured programming code.

Unlike natural language text, source code possesses unique characteristics that make search and retrieval more complex. It is highly structured, follows strict syntactic rules, and contains hierarchical relationships among elements such as functions, classes, loops, and conditional statements. Additionally, the meaning of code is determined not only by individual tokens but also by their logical relationships and execution flow. Traditional search approaches, which treat code as plain text, fail to capture these properties, resulting in suboptimal retrieval performance.

To overcome these challenges, research in code search has evolved toward incorporating structural and semantic understanding of code. Structural approaches focus on analyzing the syntax and hierarchy of code, while semantic approaches aim to interpret the meaning and intent behind both queries and code snippets. Despite these advancements, achieving an optimal balance between accuracy, efficiency, and interpretability remains a significant challenge, motivating the development of more effective hybrid systems.

1.2 EVOLUTION OF CODE SEARCH SYSTEMS

The evolution of code search systems reflects the continuous effort to improve retrieval accuracy and usability in response to the growing complexity of software systems. Initially, code search relied on traditional information retrieval techniques that treated source code as plain text. These keyword-based systems utilized methods such as TF-IDF and BM25 to match user queries with code based on term frequency and statistical relevance. While these approaches were efficient and easy to implement, they lacked the ability to understand the meaning or context of queries, often resulting in irrelevant or incomplete search results. ^{[1],[7],[11]}

To address these limitations, structural code search techniques were introduced, focusing on the syntactic organization of source code. These methods utilize representations such as Abstract Syntax Trees to decompose code into hierarchical components, including functions, classes, and control structures. By capturing the structural relationships within code, these systems are able to provide more precise and context-aware retrieval. For example, a query related to a specific function can be matched directly with function definitions rather than entire files, improving both relevance and efficiency. However, structural approaches still face challenges in interpreting natural language queries, as they primarily focus on code syntax rather than semantic meaning.

The introduction of deep learning has marked a significant advancement in code search systems. Neural network-based approaches use embeddings to represent both queries and code in a shared vector space, enabling semantic matching beyond simple keyword overlap. Techniques such as recurrent neural networks, attention mechanisms, and transformer-based models allow these systems to capture contextual and semantic relationships between words and code tokens. As a result, deep learning-based systems can better understand developer intent and provide more relevant search results.

Despite their advantages, these systems are not without limitations. They require large annotated datasets for training, involve high computational costs, and often function as black-box models with limited interpretability. Furthermore, they struggle with cross-domain adaptability, where models trained on one dataset may not perform well on different codebases due to variations in vocabulary and programming styles. These challenges highlight the need for alternative approaches that combine the strengths of structural and semantic methods while maintaining efficiency and transparency. ^{[8],[12]}

1.3 PROBLEM STATEMENT

Although significant progress has been made in the field of code search, existing systems continue to face several critical challenges that limit their effectiveness in real-world applications. Traditional keyword-based approaches fail to capture the structural and semantic characteristics of source code, leading to inaccurate and irrelevant search results. These systems rely heavily on exact keyword matching, making them sensitive to variations in terminology and unable to interpret the underlying intent of user queries.

On the other hand, deep learning-based approaches attempt to address these issues by learning semantic representations of code and queries. While these methods improve retrieval accuracy, they introduce new challenges such as high computational complexity, dependence on large training datasets, and difficulty in deployment. Moreover, the lack of explainability in these models makes it difficult for users to understand why specific results are retrieved, reducing trust and usability.

Another major issue is the problem of cross-domain adaptability. Code search models are often trained on specific datasets, such as open-source repositories, and may not generalize well to other domains with different coding styles, terminologies, or programming languages. This results in reduced performance when applied to real-world scenarios involving diverse and dynamic codebases.

Additionally, existing systems often fail to provide fine-grained retrieval, returning entire files instead of specific code snippets that directly address the user's query. This increases the effort required by developers to locate relevant information within the retrieved results.

Therefore, the problem addressed in this project is to design and develop a code search system that effectively integrates structural understanding and semantic matching while ensuring efficiency, scalability, and explainability. The system should be capable of handling multi-language repositories, providing precise snippet-level retrieval, and delivering transparent ranking results that enhance user trust and usability.

1.4 MOTIVATION

The motivation for this project is driven by the practical challenges faced by developers in modern software development environments. As software systems grow in size and complexity, developers are required to work with large codebases that often span multiple programming languages and domains. In such scenarios, efficiently locating relevant code becomes a time-consuming and challenging task. Developers frequently rely on manual search methods or basic keyword-based tools, which often fail to provide accurate and context-aware results.

One of the key motivations is to improve developer productivity by reducing the time and effort required to search for relevant code. Efficient code search systems can significantly accelerate development by enabling quick access to reusable implementations, debugging solutions, and learning resources. This is particularly important in collaborative and open-source environments, where developers interact with unfamiliar codebases.

Another important motivation is the need for systems that understand both the structure and semantics of code. While structural approaches provide insight into the organization of code, semantic techniques help capture the intent behind user queries. Combining these approaches can lead to more accurate and meaningful search results.

The lack of explainability in existing systems also serves as a strong motivation for this work. Developers need to understand why a particular result is retrieved, especially when evaluating code for correctness or suitability. An explainable system can provide insights into the factors influencing the ranking of results, thereby improving transparency and user trust.

Furthermore, the increasing use of multi-language repositories necessitates the development of systems that can operate across different programming languages. A unified solution that supports multiple languages can significantly enhance usability and applicability in real-world scenarios.

Finally, there is a need for lightweight and efficient solutions that can be deployed without requiring extensive computational resources. This project aims to address this requirement by designing a system that balances accuracy, efficiency, and practicality, making it suitable for both academic and real-world applications.

1.5 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and implement a hybrid multi-language code-aware search engine with explainable ranking that effectively addresses the limitations of existing code search systems. The proposed system aims to bridge the gap between natural language queries and structured source code by combining both structural and semantic techniques to improve retrieval accuracy and usability.

One of the key objectives is to develop a system capable of understanding the structural components of source code, such as functions, classes, methods, and control flow constructs. By leveraging parsing techniques, the system aims to preserve the hierarchical and logical relationships within code, enabling more precise and context-aware retrieval compared to traditional text-based methods.

Another important objective is to support multiple programming languages, including Python, Java, C, and C++, to ensure the system's applicability across diverse software development environments. This multi-language capability allows the system to handle heterogeneous repositories and improves its relevance in real-world scenarios.

The project also aims to implement snippet-level indexing and retrieval, where large code files are decomposed into smaller, meaningful units such as functions and classes. This approach enhances retrieval precision by returning only the most relevant portions of code rather than entire files, thereby reducing the effort required by developers to locate useful information.

A significant objective of the system is to design a hybrid ranking mechanism that combines structural features, such as identifiers and function names, with semantic similarity measures derived from embeddings. This combination ensures that the system captures both the syntactic and semantic aspects of code, leading to improved search performance.

Furthermore, the project aims to incorporate an explainable ranking mechanism, which provides transparency by indicating the factors contributing to the ranking of each result. This feature enhances user trust and allows developers to better understand the relevance of retrieved code snippets.

Finally, the system is designed to be efficient, scalable, and practical, with low latency and minimal computational requirements. The performance of the system is evaluated using standard information retrieval metrics such as Precision@K, Recall@K, Mean Reciprocal Rank (MRR), and NDCG, ensuring a comprehensive assessment of its effectiveness.

1.6 SCOPE OF THE PROJECT

The scope of this project is carefully defined to ensure a balance between functionality, feasibility, and practical applicability. The proposed system focuses on developing a hybrid code search engine that integrates structural and semantic techniques to improve code retrieval accuracy while maintaining efficiency and interpretability.

The system supports multiple programming languages, including Python, Java, C, and C++, enabling it to operate across diverse codebases. It is designed to process real-world repositories by dynamically loading code from platforms such as GitHub, allowing users to search within actual software projects rather than static datasets. This capability enhances the practicality and relevance of the system.

Within the system, source code is analyzed using parsing techniques to extract structural components such as functions, classes, and methods. These components are further divided into smaller units through snippet extraction, enabling fine-grained indexing and retrieval. This approach ensures that the system provides precise and relevant results tailored to user queries.

The project also includes the implementation of a hybrid ranking mechanism, combining structural features and semantic similarity to improve retrieval performance. Additionally, an explainable ranking system is incorporated to provide transparency in the decision-making process, allowing users to understand why specific results are returned.

However, the scope of the project is limited in certain aspects. The system does not aim to implement large-scale deep learning models requiring extensive training or high computational resources. It is not intended to replace industrial-level search engines but rather to provide a lightweight and efficient alternative suitable for academic and practical use. Furthermore, the system primarily focuses on publicly available repositories and does not address highly domain-specific or proprietary codebases.

Overall, the scope of the project ensures that the system remains focused, implementable, and aligned with real-world requirements while maintaining a balance between complexity and performance.

1.7 KEY CONTRIBUTIONS OF THE WORK

This project makes several significant contributions to the field of code search by addressing the limitations of existing approaches and introducing a balanced, practical solution. One of the primary contributions is the development of a hybrid code-aware search engine that integrates both structural and semantic techniques. This approach enables the system to capture the hierarchical organization of code while also understanding the intent behind user queries, resulting in improved retrieval accuracy.

Another important contribution is the implementation of multi-language support, allowing the system to process and analyze code written in different programming languages. This enhances the versatility and applicability of the system, making it suitable for diverse development environments and real-world scenarios.

The project also introduces snippet-level indexing and retrieval, which allows the system to return precise code segments such as functions and classes rather than entire files. This significantly improves usability by reducing the effort required by developers to locate relevant information within large codebases.

A key innovation of this work is the incorporation of an explainable ranking mechanism, which provides transparency by clearly indicating the factors contributing to the ranking of results. This feature distinguishes the system from many existing approaches, particularly deep learning-based models, which often operate as black boxes.

Additionally, the system integrates real-time repository processing, enabling users to search code from live GitHub repositories. This enhances the practicality of the system and ensures that it remains relevant in dynamic development environments.

Finally, the project demonstrates that it is possible to achieve effective code search without relying on heavy deep learning models by designing a lightweight and efficient system. This contribution highlights the importance of balancing performance, interpretability, and computational efficiency, making the proposed approach suitable for both academic research and practical applications.

1.8 ORGANIZATION OF THE REPORT

This report is systematically organized into nine well-defined chapters, each addressing a specific component of the project to ensure a logical progression from conceptual understanding to implementation and evaluation. The structure is designed to provide clarity, coherence, and a comprehensive understanding of the proposed hybrid code-aware search system. Each chapter builds upon the previous one, enabling the reader to follow the development of the system from problem identification to final conclusions.

Chapter 1

This chapter serves as the introductory foundation of the report, presenting the background of code search systems and emphasizing their importance in modern software development. It discusses the challenges associated with traditional and contemporary approaches, formulates the problem statement, and outlines the motivation behind the project. Additionally, it defines the objectives, scope, and key contributions, thereby establishing the context and necessity for the proposed solution.

Chapter 2

This chapter focuses on an in-depth literature review and research analysis. It explores various existing techniques in code search, including keyword-based information retrieval methods, structural approaches such as Abstract Syntax Tree (AST) analysis, and advanced deep learning-based models. This chapter also examines the base paper on cross-domain code search and critically analyzes its methodology and limitations. By comparing multiple approaches, it identifies significant research gaps such as semantic mismatch, lack of explainability, and domain dependency, which directly influence the design of the proposed system.

Chapter 3

This chapter provides a detailed analysis of existing code search systems and their architectures. It explains the working of different modules such as data collection, preprocessing, indexing, query processing, and ranking. The chapter critically evaluates the shortcomings of these systems, including their inability to capture structural and semantic relationships, limited adaptability, and lack of transparency in result ranking. This analysis justifies the need for an improved hybrid approach.

Chapter 4

This chapter presents the proposed hybrid code-aware search system in a comprehensive manner. It describes the overall system architecture and explains the workflow from input to output. Each module, including GitHub repository loading, multi-language parsing, snippet extraction, query processing, structural ranking, semantic similarity computation, and explainable ranking, is discussed in detail. The chapter highlights how the integration of these components results in improved accuracy, efficiency, and usability compared to existing methods.

Chapter 5

This chapter introduces the mathematical model and algorithmic design of the system. It formulates the problem in a structured manner and explains the hybrid scoring mechanism used to rank code snippets. The chapter provides detailed descriptions of the algorithms used for structural and semantic similarity computation. It also defines the evaluation metrics, including Precision@K, Recall@K, Mean Reciprocal Rank (MRR), NDCG, and latency, which are essential for assessing the performance of the system.

Chapter 6

This chapter focuses on the implementation details of the proposed system. It describes the technologies and tools used, including the backend framework, frontend interface, and supporting libraries. The chapter explains how individual modules are developed and integrated to form a complete working system. It also discusses practical considerations such as efficiency, scalability, and real-time processing.

Chapter 7

This chapter presents the experimental setup and results obtained from testing the system. It describes the dataset used, the evaluation methodology, and the performance metrics. The results are analyzed using tables and graphical representations to demonstrate the effectiveness of the hybrid approach. This chapter highlights how the proposed system performs in terms of accuracy, relevance, and response time.

Chapter 8

This chapter provides a detailed discussion of the system’s performance, including its strengths and limitations. It evaluates the practical applicability of the system and compares it with existing approaches and the base paper. This chapter also discusses trade-offs between accuracy, efficiency, and explainability, providing a deeper understanding of the system’s behavior in real-world scenarios.

Chapter 9

Finally, this chapter concludes the report by summarizing the key findings and contributions of the project. It highlights how the proposed system addresses the challenges identified earlier and emphasizes its practical significance. The chapter also outlines potential future enhancements, such as extending support to additional programming languages, integrating advanced semantic models, and improving scalability for large-scale deployment.

CHAPTER II

LITERATURE REVIEW

2.1 OVERVIEW OF CODE SEARCH TECHNIQUES

Code search techniques have evolved significantly over the past decade in response to the growing scale and complexity of software repositories. With the increasing reliance on open-source platforms and collaborative development, the need for efficient and accurate code retrieval systems has become critical. Code search techniques aim to enable developers to locate relevant code snippets based on natural language queries or code-based inputs, thereby improving productivity and facilitating software reuse.

Broadly, code search techniques can be categorized into three major types: traditional information retrieval-based methods, structural approaches, and semantic or deep learning-based methods. Traditional methods focus on keyword matching and statistical relevance, while structural approaches analyze the syntactic organization of code using representations such as Abstract Syntax Trees. More recent approaches leverage machine learning and deep learning techniques to capture the semantic relationships between queries and code.

Each category offers distinct advantages and limitations. Keyword-based methods are efficient but lack semantic understanding, structural approaches provide better representation of code organization but struggle with natural language queries, and deep learning-based models offer improved semantic matching but introduce challenges such as computational complexity and lack of explainability. As a result, current research trends are moving toward hybrid approaches that combine multiple techniques to achieve a balance between accuracy, efficiency, and interpretability.

2.2 TRADITIONAL INFORMATION RETRIEVAL METHODS

Traditional code search systems are primarily based on information retrieval techniques originally developed for text documents. In these systems, source code is treated as plain text and processed using methods such as tokenization, normalization, and indexing. Common algorithms used in this category include Term Frequency–Inverse Document Frequency (TF-IDF) and Best Matching 25 (BM25), which rank documents based on the frequency and importance of terms within the code and the query. ^{[1],[7]}

These methods are computationally efficient and easy to implement, making them suitable for large-scale repositories. They enable fast retrieval by building inverted indices that allow quick lookup of relevant documents. However, the effectiveness of these approaches is limited by their

reliance on exact keyword matching. They do not account for the structure or semantics of code, leading to significant challenges in accurately interpreting user queries.

One of the major issues associated with traditional methods is the vocabulary mismatch problem, where different terms are used to describe the same concept. For example, a query for “array sorting” may not match code containing the term “arrange list,” even though both represent similar functionality. Additionally, these systems cannot distinguish between different contexts in which a keyword appears, resulting in irrelevant or ambiguous results.

Furthermore, traditional approaches fail to capture the hierarchical nature of source code, such as the relationships between functions, classes, and modules. As a result, they often return entire files rather than specific code snippets, increasing the effort required by developers to locate relevant information. Despite these limitations, traditional methods remain widely used due to their simplicity and efficiency, often serving as a baseline for comparison with more advanced techniques.

2.3 STRUCTURAL CODE SEARCH APPROACHES (AST-BASED)

To address the limitations of keyword-based methods, structural code search approaches were introduced, focusing on the syntactic organization of source code. These methods utilize representations such as Abstract Syntax Trees to capture the hierarchical structure and logical relationships within code. An AST represents source code as a tree, where each node corresponds to a syntactic construct such as a function, class, or control statement.^{[9],[11]}

By analyzing the structure of code, AST-based approaches can extract meaningful components such as function definitions, class hierarchies, and method calls. This enables more precise indexing and retrieval, as the system can focus on specific code elements rather than treating the entire file as a single unit. For example, a query related to a particular function can be matched directly with function definitions, improving both relevance and efficiency.

Structural approaches also allow for the identification of relationships between different parts of code, such as dependencies and control flow. This provides a deeper understanding of code compared to traditional text-based methods. Additionally, snippet-level extraction enables fine-grained retrieval, allowing users to obtain only the relevant portions of code instead of entire files.

However, despite their advantages, structural approaches have certain limitations. They primarily focus on syntax rather than semantics, meaning they cannot fully interpret the intent behind user queries. For instance, while they can identify a function structure, they may not understand the purpose or functionality of the code. Furthermore, these approaches require language-specific parsing techniques, making it challenging to support multiple programming languages within a single system.

Nevertheless, structural methods play a crucial role in modern code search systems and are often combined with semantic techniques to improve overall performance.

2.4 DEEP LEARNING-BASED CODE SEARCH MODELS

Deep learning–based code search models represent a significant advancement in the field by addressing the semantic gap between natural language queries and programming code. These models use neural networks to learn representations of both queries and code in a shared vector space, enabling similarity-based matching beyond simple keyword overlap.

One of the key techniques used in these models is embedding, where words, sentences, and code snippets are converted into numerical vectors that capture their semantic meaning. Methods such as Word2Vec, fastText, and sentence transformers are commonly used for this purpose. These embeddings allow the system to recognize relationships between different terms, even when they are not lexically identical.

Advanced models incorporate architectures such as recurrent neural networks, convolutional neural networks, and transformer-based models with attention mechanisms. These architectures enable the system to capture contextual information and relationships between different parts of the input. For example, attention mechanisms allow the model to focus on the most relevant parts of a query or code snippet, improving matching accuracy. ^{[6],[8],[10]}

Deep learning–based approaches have demonstrated significant improvements in code search performance, particularly in handling complex queries and capturing semantic relationships. However, they also introduce several challenges. These models require large annotated datasets for training, which may not always be available. They are computationally expensive, requiring significant processing power and memory. Additionally, they often function as black-box models, providing limited insight into how results are generated, which reduces transparency and user trust.

Another important limitation is their domain dependency, where models trained on specific datasets may not generalize well to different codebases or programming languages. This issue is particularly relevant in real-world scenarios involving diverse and dynamic repositories.

Despite these challenges, deep learning–based models continue to play a significant role in advancing code search research. Many modern systems adopt hybrid approaches that combine deep learning techniques with structural and traditional methods to achieve a balance between accuracy, efficiency, and interpretability.

2.5 CROSS-DOMAIN CODE SEARCH (BASE PAPER ANALYSIS)

Cross-domain code search has emerged as a critical research area due to the increasing diversity of software repositories and programming practices. Traditional and deep learning-based code search models are typically trained on specific datasets, such as open-source repositories, and perform well within those domains. However, when applied to different domains with varying coding styles, terminologies, and structures, their performance often degrades significantly. This limitation highlights the importance of developing models that can generalize across domains.

The base paper, titled “*An Adaptive Model for Cross-Domain Code Search*,” addresses this challenge by proposing a deep learning-based approach designed to improve adaptability across different datasets. The model focuses on learning transferable features that can be applied to new domains without requiring extensive retraining. One of the key ideas presented in the paper is that while vocabulary may differ across domains, the underlying grammatical and structural patterns of code remain relatively consistent. By leveraging this insight, the model aims to improve cross-domain performance.^[1]

The proposed approach in the base paper integrates several advanced components, including embedding techniques, self-attention mechanisms, and sequence modeling using recurrent neural networks such as Long Short-Term Memory (LSTM). The embedding layer converts both queries and code into vector representations, enabling semantic matching. The self-attention mechanism captures relationships between different parts of the input, allowing the model to focus on relevant features. The LSTM component further enhances the model’s ability to learn sequential patterns and grammar structures in code.

A key feature of the model is the use of a word matching matrix, which computes similarity between query words and code tokens. This enables fine-grained comparison and improves the accuracy of retrieval. The final ranking of code snippets is determined using similarity measures such as cosine similarity, ensuring that the most relevant results are prioritized.

Although the base paper demonstrates improved performance compared to existing models, it has certain limitations. The model requires substantial computational resources and training data, making it difficult to deploy in resource-constrained environments. Additionally, the lack of explainability in the ranking process reduces transparency, making it challenging for users to understand how results are generated. These limitations highlight the need for alternative approaches that provide both efficiency and interpretability.

2.6 SEMANTIC GAP AND OUT-OF-VOCABULARY (OOV) PROBLEM

One of the most significant challenges in code search is the semantic gap between natural language queries and programming code. Developers typically express their requirements in natural language, while code is written in structured programming languages with specific syntax and semantics. This difference creates a disconnect between the way queries are formulated and how code is represented, making it difficult for search systems to accurately match the two.

The semantic gap arises because similar concepts can be expressed using different words or structures in code. For example, a query such as “read file” may correspond to code using functions like `open()` or `FileReader`, which do not explicitly contain the query terms. Traditional keyword-based systems fail to capture such relationships, resulting in poor retrieval performance.

Deep learning-based approaches attempt to address this issue by learning semantic representations of queries and code. However, these methods introduce another challenge known as the Out-of-Vocabulary (OOV) problem. OOV occurs when a model encounters words or tokens that were not present in the training dataset. In the context of code search, this is particularly problematic because different domains often use unique variable names, function names, and domain-specific terminology.

The base paper highlights that OOV is a major factor affecting cross-domain performance. When a model is trained on one dataset and applied to another, it may fail to recognize new terms, leading to reduced accuracy. Although embedding techniques such as fastText attempt to mitigate this issue by capturing subword information, they cannot completely eliminate the problem.

Addressing the semantic gap and OOV problem remains a key challenge in code search research. Effective solutions require a combination of structural understanding, semantic representation, and adaptability to new domains. This motivates the development of hybrid systems that can leverage multiple techniques to improve performance.

2.7 COMPARATIVE STUDY OF EXISTING SYSTEMS

A comparative analysis of existing code search systems reveals significant differences in their approaches, capabilities, and limitations. Traditional keyword-based systems are simple and efficient but lack semantic understanding, resulting in poor retrieval accuracy. Structural approaches improve relevance by analyzing code syntax and hierarchy but are limited in their ability to interpret natural language queries.

Deep learning-based models, such as DeepCS and CodeHow, provide improved semantic matching by representing queries and code in a shared vector space. These models use techniques such as embeddings and attention mechanisms to capture contextual relationships. However, they

require large datasets, high computational resources, and often lack transparency in their ranking processes. [2],[3],[4],[5]

The base paper’s adaptive model (ACD-SA) further enhances performance by addressing cross-domain challenges through self-attention and sequence modeling. While it achieves higher accuracy, it remains complex and resource-intensive, making it less practical for real-world deployment. [1],[12],[13]

In contrast, the proposed system in this project adopts a hybrid approach that combines structural and semantic techniques while maintaining efficiency and explainability. Unlike deep learning models, it does not rely on heavy training processes and instead uses lightweight similarity methods. Additionally, it incorporates an explainable ranking mechanism, which provides transparency and improves user trust.

This comparison highlights that no single approach is sufficient to address all challenges in code search. Hybrid systems offer a promising direction by integrating the strengths of different methods while minimizing their limitations.

2.8 RESEARCH GAPS IDENTIFIED

Based on the analysis of existing literature and systems, several key research gaps have been identified. One of the primary gaps is the lack of systems that effectively combine structural and semantic understanding of code. While traditional methods focus on text-based matching and structural approaches emphasize syntax, there is a need for integrated solutions that address both aspects simultaneously.

Another significant gap is the lack of explainability in modern code search systems. Many deep learning-based models operate as black boxes, providing little insight into how results are ranked. This reduces user trust and makes it difficult for developers to validate retrieved code snippets.

The issue of cross-domain adaptability also remains unresolved. Existing models often perform well within specific datasets but fail when applied to different domains due to variations in vocabulary and coding styles. This limitation highlights the need for more adaptable and generalizable systems.

Additionally, many systems lack efficiency and scalability, particularly those based on deep learning techniques. High computational requirements make them unsuitable for real-time applications and resource-constrained environments.

Finally, there is a need for fine-grained retrieval mechanisms that provide snippet-level results rather than entire files. This would improve usability by allowing developers to directly access relevant code segments without additional effort.

These research gaps form the basis for the proposed system, which aims to provide a hybrid, efficient, and explainable solution for code search. By addressing these limitations, the project contributes to the advancement of practical and effective code search techniques.

Table 2.1: Comparison Between Base Paper and Proposed System

Aspect	Base Paper	Our Proposed Work
Main Focus	Cross-domain semantic code search	Code-aware search with explainable ranking
Approach	Deep learning-based adaptive model	Lightweight, structured search system
Code Understanding	Token embeddings and learned representations	AST-based structural analysis
Explainability	Not supported (black-box ranking)	Clear explanation of ranking decisions
Model Complexity	High (self-attention + LSTM)	Low to moderate (no heavy DL models)
Practical Usability	Research-oriented	Developer-oriented and interpretable
Evaluation Metrics	MRR, MAP, Hit@K	Precision@K, Recall@K, MRR, Response Time

Table 2.2: Literature Review of Code Search Techniques

No.	Paper Title	Journal/Conference	Methodology / Technique	Advantages	Limitations	Research Gap Identified
1	An adaptive model for cross-domain code search	Information and Software Technology (Elsevier 2025)	Adaptive semantic ranking for code search across domains	Strong retrieval performance across diverse repositories	Model complexity, limited explainability	Does not provide transparent ranking explanations

2	Code search engines for the next generation	Journal of Systems and Software (Elsevier) 2024	Survey of modern code search systems and challenges	Clearly identifies limitations of keyword-based search	Numerous methodologies Without clear proof of work	Lacks practical, explainable system design
3	Code semantic enrichment for deep code search	Journal of Systems and Software (Elsevier) 2024	Semantic feature enrichment for code representations	Improves semantic relevance	High computational cost	Explainability not addressed
4	Query-oriented two-stage attention-based model for code search	Journal of Systems and Software (Elsevier) 2024	Query-aware attention-based ranking	Better query-code alignment	Black-box deep model	No transparency in ranking decisions
5	A mutual embedded self-attention network model for code search	Journal of Systems and Software (Elsevier) 2023	Self-attention-based semantic similarity learning	Captures deep semantic relationships	Difficult to interpret	Lack of explainable output
6	Big Code Search: A Bibliography	ACM Computing Surveys 2023	Systematic survey of code search literature	Comprehensive taxonomy of code search	No clear Proof of work	Identifies need for explainable and structured search

7	A systematic literature review on deep learning-based code search	Empirical Software Engineering (Springer) 2022	Review of DL-based code search methods	Clear classification of methods	Focuses mainly on deep learning	Limited discussion on structural explainability
8	Semantic code search: Challenges, techniques, and future directions	Journal of Software: Evolution and Process (Wiley) 2022	Analysis of challenges in semantic code search	Identifies open research challenges	Unexplored implementation	Need for practical, developer-oriented tools
9	Improving code search using structural and semantic information	Software Quality Journal (Springer) 2021	Structural + semantic code feature extraction	Shows benefit of structure-aware search	Limited dataset	No ranking explanation mechanism
10	CodeGRU: Context-aware deep learning model for code search	Information and Software Technology (Elsevier) 2020	GRU-based contextual code modeling	Improves context understanding	Computationally heavy	Lack of explainability and transparency
11	Neural Code Search: A Survey	ICSE (IEEE/ACM Conference) 2021	Survey of neural code search techniques	Covers state-of-the-art methods	Numerous methodologies without proper Proof of work	Highlights interpretability issues

12	Representation learning for code search	FSE (ACM Conference) 2022	Code embedding and representation learning	Effective semantic matching	Requires large datasets	Explainability not considered
13	Multi-stage code search with neural reranking	SIGIR (ACM Conference) 2023	BM25 + neural reranking pipeline	Improves ranking accuracy	Complex pipeline	No transparent ranking logic
14	An empirical study of code search in intelligent IDEs	ASE (IEEE Conference) 2024	Empirical analysis of developer search behavior	Strong practical relevance	No algorithmic proposal with Proof	Need for better developer-oriented search tools

CHAPTER III

EXISTING SYSTEM

3.1 OVERVIEW OF EXISTING SYSTEMS

Code search systems have become an essential component in modern software development, enabling developers to efficiently locate relevant code snippets from large repositories. Existing systems are primarily designed to retrieve code based on user queries, which may be expressed either as keywords or natural language descriptions. Over time, these systems have evolved from simple keyword-based approaches to more advanced methods incorporating structural and semantic analysis.

Most existing systems can be broadly categorized into three types: keyword-based systems, structural code search systems, and deep learning-based systems. Keyword-based systems rely on text matching techniques to identify relevant code, while structural systems analyze the syntax and organization of code using representations such as Abstract Syntax Trees. Deep learning-based systems, on the other hand, use neural networks to capture semantic relationships between queries and code.

Despite these advancements, existing systems still face significant challenges in accurately retrieving relevant code. Many systems fail to effectively bridge the gap between natural language queries and programming code, resulting in irrelevant or incomplete results. Additionally, the lack of explainability and adaptability further limits their usability in real-world scenarios.

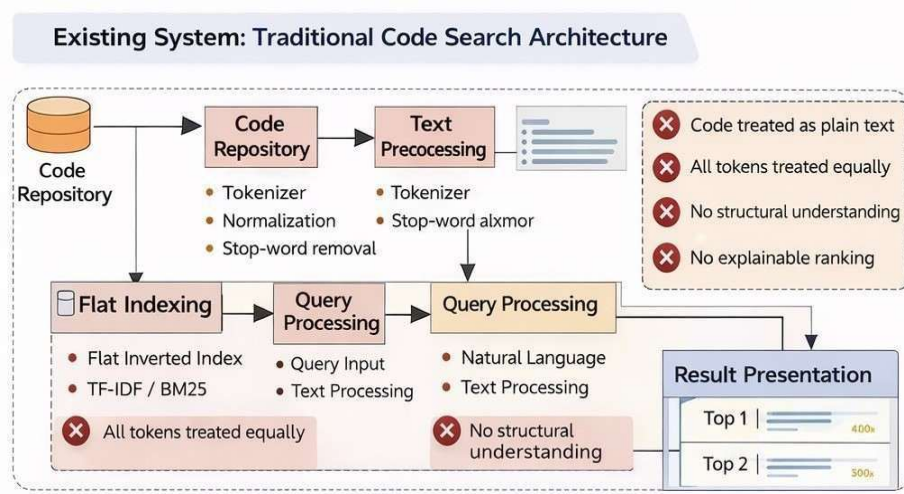


Figure 3.1: Existing System Architecture Diagram

3.2 ARCHITECTURE OF TRADITIONAL CODE SEARCH SYSTEMS

The architecture of traditional code search systems typically follows a pipeline-based approach consisting of several stages, including data collection, preprocessing, indexing, query processing, and result retrieval. Each stage plays a crucial role in determining the overall performance and efficiency of the system.

In the data collection stage, source code is gathered from repositories and stored in a structured format. This is followed by preprocessing, where the code is tokenized, cleaned, and normalized to prepare it for indexing. The indexing stage involves creating data structures such as inverted indices that allow for efficient retrieval of code based on keywords.

During query processing, the user's input is analyzed and transformed into a format suitable for matching against the indexed data. The system then applies similarity measures to compare the query with stored code snippets and ranks the results based on relevance. Finally, the retrieved results are presented to the user, often in the form of code files or snippets.

While this architecture is efficient and widely used, it primarily focuses on textual similarity and does not incorporate structural or semantic understanding of code. This limitation significantly affects the accuracy and usefulness of the retrieved results.

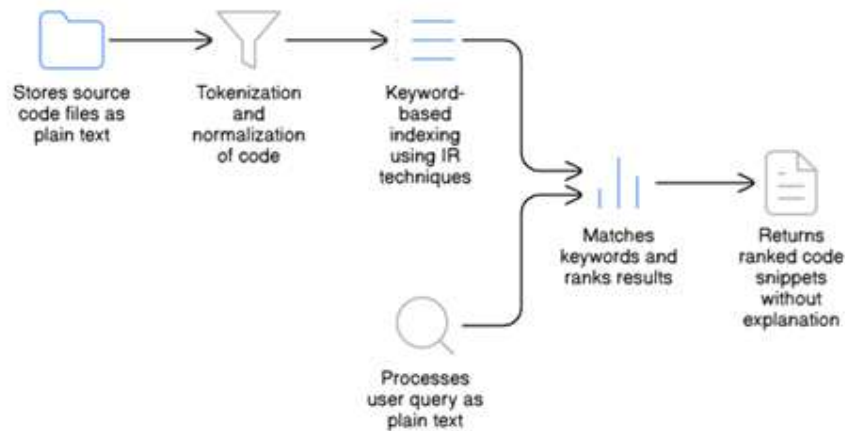


Figure 3.2: Existing System Workflow Diagram

3.3 MODULES OF EXISTING SYSTEM

Existing code search systems are composed of several functional modules that work together to perform retrieval tasks. These modules are designed to process code and queries in a sequential manner, enabling the system to generate relevant results.

Table 3.1: Modules in the Existing System

Module	Description
Code Repository Module	Stores source code from datasets
Preprocessing Module	Tokenization and normalization
Indexing Module	Builds inverted index
Query Processing Module	Processes user queries
Similarity Matching	TF-IDF/BM25 matching
Result Module	Displays results

3.3.1 CODE REPOSITORY MODULE

The code repository module is responsible for collecting and storing source code from various repositories. It acts as the primary data source for the system and provides the raw input for further processing. In most traditional systems, repositories are stored locally or accessed through predefined datasets. However, these systems often rely on static data, limiting their ability to handle dynamic and real-time code updates.

3.3.2 TEXT-BASED PREPROCESSING

The preprocessing module prepares the source code for indexing by performing operations such as tokenization, stop-word removal, and normalization. In traditional systems, code is treated as plain text, and preprocessing focuses on extracting keywords and identifiers. While this step improves retrieval efficiency, it does not capture the structural or semantic aspects of code.

3.3.3 FLAT INDEXING

The indexing module creates data structures that enable efficient search and retrieval. In traditional systems, indexing is performed at the file level, where entire code files are treated as individual documents. This approach simplifies the indexing process but reduces retrieval precision, as users often need to manually search within large files to find relevant code.

3.3.4 QUERY PROCESSING

The query processing module analyzes the user's input and transforms it into a suitable representation for matching. In keyword-based systems, this involves extracting terms and matching them against indexed keywords. However, the lack of semantic understanding often leads to mismatches between queries and code.

3.3.5 SIMILARITY MATCHING AND RANKING

The similarity matching module compares the processed query with indexed code using techniques such as TF-IDF or BM25. Results are ranked based on their similarity scores, with higher-ranked results considered more relevant. While this approach is efficient, it relies heavily on lexical similarity and does not account for code semantics.

3.3.6 RESULT PRESENTATION

The result presentation module displays the retrieved code to the user. In most systems, results are presented as a list of files or snippets ranked by relevance. However, the lack of explanation for ranking decisions makes it difficult for users to understand why certain results are returned.

3.4 LIMITATIONS OF EXISTING SYSTEMS

Despite their widespread use, existing code search systems suffer from several limitations that reduce their effectiveness. One of the primary issues is the reliance on keyword-based

matching, which fails to capture the semantic meaning of queries and code. This leads to inaccurate results, particularly in cases where different terms are used to describe the same concept.

Another major limitation is the lack of structural understanding. Traditional systems treat code as plain text and ignore its hierarchical organization, resulting in poor representation of relationships between different components. This limits the system’s ability to provide context-aware results.

Deep learning-based systems, while addressing semantic challenges, introduce new issues such as high computational cost, dependency on large datasets, and lack of explainability. These systems often operate as black boxes, making it difficult for users to understand how results are generated. [6],[8],[12]

Additionally, existing systems struggle with cross-domain adaptability, where models trained on specific datasets perform poorly when applied to different repositories. This is further compounded by the Out-of-Vocabulary problem, where new terms are not recognized by the system. [1],[7],[11]

Finally, most systems provide file-level retrieval rather than snippet-level results, requiring users to manually search within files. This reduces efficiency and increases the effort required to locate relevant code.

3.5 NEED FOR PROPOSED SYSTEM

The limitations of existing code search systems highlight the need for an improved approach that addresses both structural and semantic challenges while maintaining efficiency and usability. A system that combines multiple techniques can overcome the weaknesses of individual approaches and provide more accurate and meaningful results.

The proposed system aims to address these issues by introducing a hybrid approach that integrates structural analysis with semantic similarity. By extracting code snippets and analyzing their structure, the system can provide fine-grained and context-aware retrieval. At the same time, semantic techniques enable better matching between natural language queries and code.

Another key requirement is the inclusion of an explainable ranking mechanism, which provides transparency and helps users understand why specific results are retrieved. This feature is particularly important for building trust and improving usability.

Furthermore, the system is designed to support multiple programming languages and real-time repository processing, ensuring its applicability in modern development environments. By focusing on efficiency and practicality, the proposed system provides a lightweight alternative to complex deep learning models.

Overall, the proposed system addresses the key limitations of existing approaches and offers a balanced solution that improves accuracy, transparency, and usability in code search.

CHAPTER IV

PROPOSED SYSTEM

4.1 OVERVIEW OF PROPOSED SYSTEM

The proposed system is a Hybrid Multi-Language Code-Aware Search Engine with Explainable Ranking, designed to overcome the limitations of traditional and deep learning–based code search systems. The system integrates both structural and semantic techniques to improve the accuracy, efficiency, and interpretability of code retrieval. Unlike conventional approaches that rely solely on keyword matching or complex neural networks, the proposed system adopts a balanced approach that combines lightweight processing with meaningful analysis of code.

The system is capable of processing real-world repositories by dynamically accepting GitHub URLs and extracting source code files across multiple programming languages. It supports languages such as Python, Java, C, and C++, enabling it to operate in diverse development environments. By leveraging language-specific parsing techniques, the system analyzes the structure of code and extracts meaningful components such as functions, classes, and methods.

A key feature of the proposed system is its snippet-level retrieval mechanism, where large code files are decomposed into smaller, logically independent units. This allows the system to provide precise and relevant results directly aligned with user queries, reducing the need for manual searching within files. Additionally, the system incorporates a hybrid ranking approach that combines structural features and semantic similarity, ensuring that both the organization and meaning of code are considered during retrieval.

Another important aspect of the system is its explainable ranking mechanism, which provides transparency by indicating the factors contributing to the ranking of results. This enhances user trust and usability, distinguishing the system from many existing approaches that lack interpretability. Overall, the proposed system offers a practical, efficient, and scalable solution for modern code search challenges.

4.2 DESIGN GOALS

The design of the proposed system is guided by several key objectives aimed at addressing the limitations of existing code search systems while ensuring practical usability. One of the primary goals is to achieve high retrieval accuracy by effectively matching user queries with relevant code snippets. This is accomplished by combining structural analysis with semantic similarity, allowing the system to capture both the syntax and meaning of code.

Another important design goal is efficiency, ensuring that the system can process queries and return results with minimal latency. Unlike deep learning–based models that require extensive

computation, the proposed system is designed to operate using lightweight techniques that provide fast and reliable performance.

The system also emphasizes explainability, which is often lacking in modern code search solutions. By providing clear insights into the ranking process, the system allows users to understand why specific results are retrieved, improving transparency and trust.

Scalability is another critical design consideration, as the system must handle large repositories and multiple programming languages. The use of modular architecture and efficient indexing techniques ensures that the system can scale effectively without significant performance degradation.

Additionally, the system is designed to be flexible and adaptable, supporting different programming languages and repository structures. This ensures that it can be applied in various development environments and use cases.

Finally, the system aims to provide a user-friendly interface, enabling developers to easily input queries and interpret results. The combination of accuracy, efficiency, explainability, scalability, and usability forms the foundation of the proposed system design.

4.3 SYSTEM ARCHITECTURE

The architecture of the proposed system follows a modular and pipeline-based design, where each component performs a specific function in the overall code search process. The system is structured to ensure efficient data flow from input to output while maintaining flexibility and scalability.

At the initial stage, the system accepts a GitHub repository URL as input. The repository loader module clones or accesses the repository and retrieves relevant source code files. These files are then passed to the file processing module, where they are filtered based on supported programming languages.

The next stage involves code parsing and analysis, where language-specific parsers are used to extract structural elements such as functions, classes, and methods. This step preserves the hierarchical organization of code and enables the system to perform fine-grained analysis.

Following parsing, the system performs snippet extraction, where code is divided into smaller units that can be independently indexed and retrieved. These snippets are stored along with metadata such as file paths, line numbers, and language information, facilitating efficient retrieval.

The query processing module handles user input by analyzing and transforming queries into a suitable representation. The system then applies a hybrid ranking mechanism, which combines structural matching with semantic similarity to evaluate the relevance of each snippet.

Finally, the results are presented to the user through an interface that displays ranked code snippets along with explanations of their relevance. This modular architecture ensures that each component can be independently optimized and extended, making the system both robust and scalable.

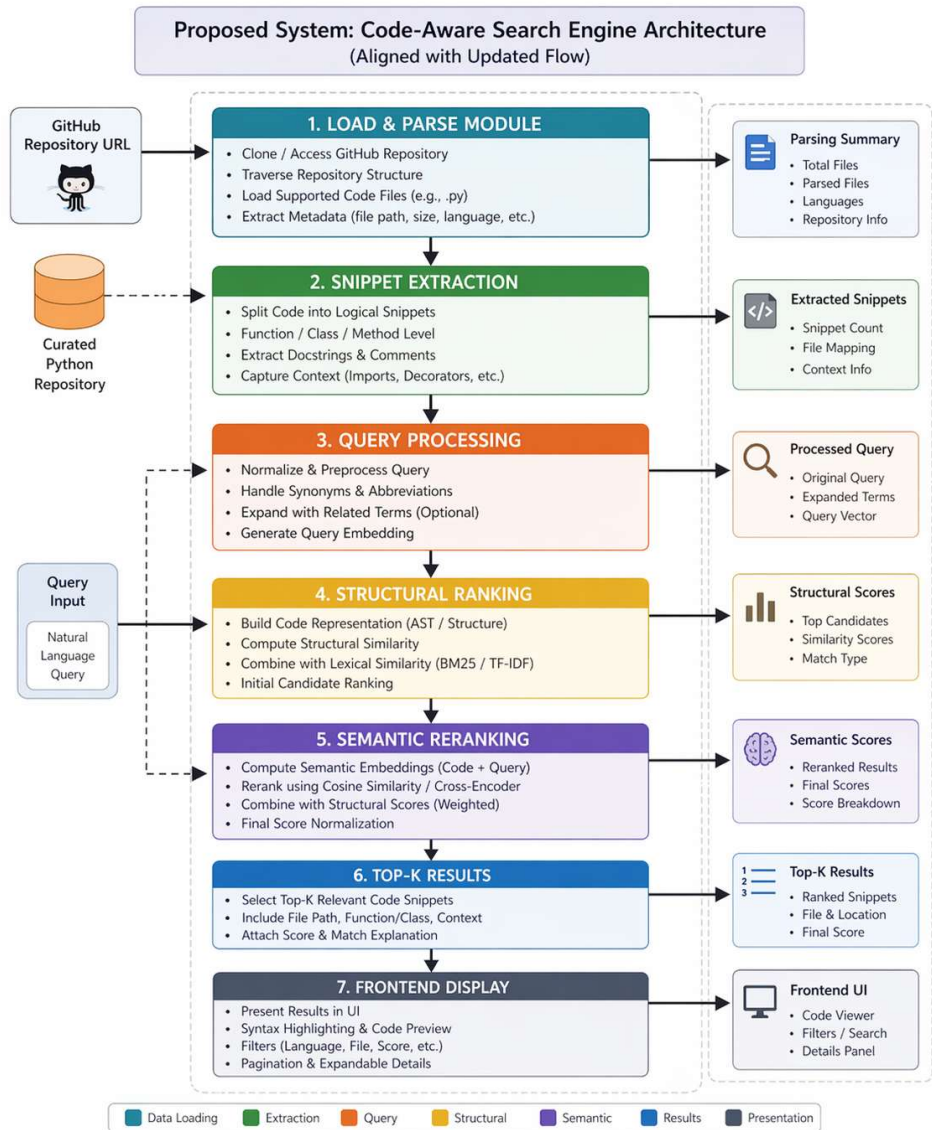


Figure 4.1: Proposed System Architecture Diagram

4.4 SYSTEM WORKFLOW

The workflow of the proposed system describes the step-by-step process through which user queries are transformed into relevant code search results. The process begins when the user provides a GitHub repository URL along with a search query. The system first retrieves the repository and extracts source code files corresponding to supported programming languages.

Once the code is collected, it undergoes preprocessing and parsing to identify structural components such as functions, classes, and methods. These components are then segmented into smaller snippets, which are indexed for efficient retrieval. This step ensures that the system can provide precise results at a granular level.

When a user submits a query, the query processing module analyzes the input and extracts key terms and semantic features. The system then compares the query with indexed code snippets using the hybrid ranking mechanism. Structural matching evaluates features such as function names and identifiers, while semantic similarity measures the contextual relevance between the query and code.

The results are ranked based on a combined score derived from both structural and semantic components. The top-ranked snippets are then presented to the user, along with explanations indicating the factors influencing their ranking. This enhances the transparency and usability of the system.

Overall, the workflow ensures a seamless and efficient transition from input to output, enabling accurate, explainable, and user-friendly code search.

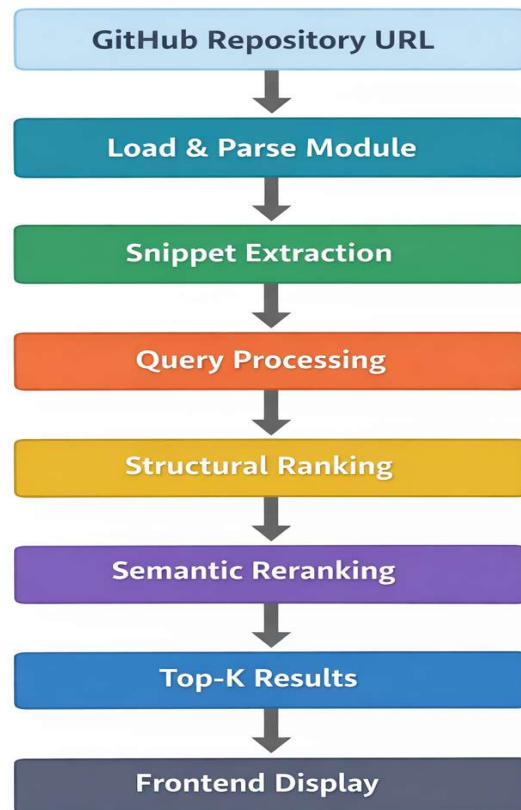


Figure 4.2: Module Interaction Diagram

4.5 DETAILED MODULE DESCRIPTION

The proposed system is composed of several interconnected modules, each responsible for performing a specific function within the code search pipeline. These modules work together to ensure efficient processing of repositories, accurate extraction of code snippets, and effective retrieval of relevant results. The modular design enhances flexibility, scalability, and ease of maintenance.

Table 4.1: Modules in the Proposed System

Module	Description
GitHub Loader	Fetches repositories
File Loader	Loads multi-language files
AST Parser	Extracts structure
Snippet Extractor	Generates snippets
Indexing	Stores indexed snippets
Query Engine	Processes queries
Hybrid Ranking	Combines scores
Explainability	Provides reasoning
UI	Displays results

4.5.1 GITHUB REPOSITORY LOADER

The GitHub repository loader module is responsible for acquiring source code from external repositories. The system accepts a repository URL as input and dynamically clones or accesses the repository using appropriate APIs or tools. This approach allows the system to work with real-world data instead of relying on static datasets.

The module ensures that only relevant files are retrieved by filtering based on supported programming languages. It also handles issues such as directory traversal and file organization, ensuring that all source files are properly collected for further processing. This dynamic data acquisition enables the system to remain up-to-date with evolving codebases.

4.5.2 MULTI-LANGUAGE FILE LOADER

Once the repository is retrieved, the multi-language file loader module processes the collected files and categorizes them based on their programming language. This module identifies file types such as .py, .java, .cpp, .c, and other relevant formats.

The system is designed to handle multiple languages by applying language-specific processing rules. This ensures that each file is parsed correctly according to its syntax and structure. By supporting multiple languages, the system becomes more versatile and applicable to a wide range of development scenarios.

4.5.3 LANGUAGE-SPECIFIC PARSING MODULE

The parsing module is responsible for analyzing the structure of source code and extracting meaningful components. Using language-specific parsing techniques, the system identifies elements such as functions, classes, methods, and control structures.

This process preserves the hierarchical organization of code, allowing the system to understand relationships between different components. Parsing plays a crucial role in enabling structural analysis, which is essential for accurate and context-aware code retrieval. It also ensures that the system can operate consistently across different programming languages.

4.5.4 SNIPPET EXTRACTION ENGINE

The snippet extraction module divides large code files into smaller, meaningful units such as functions and classes. This approach enables fine-grained indexing and retrieval, allowing users to access only the relevant portions of code rather than entire files.

Each snippet is associated with metadata, including file path, line numbers, and programming language. This information is used during retrieval to provide context and improve usability. Snippet-level extraction significantly enhances precision and reduces the effort required by developers to locate useful code.

4.5.5 QUERY PROCESSING ENGINE

The query processing module handles user input and prepares it for matching against indexed code snippets. It analyzes the query to extract key terms and semantic features, ensuring that both lexical and contextual information are captured.

The processed query is then used in the ranking stage, where it is compared with code snippets using both structural and semantic techniques. This module plays a critical role in bridging the gap between natural language queries and programming code.

4.6 HYBRID RANKING SYSTEM

The hybrid ranking system is the core component of the proposed approach, responsible for determining the relevance of code snippets based on user queries. It combines structural and semantic techniques to achieve a balance between accuracy and efficiency.

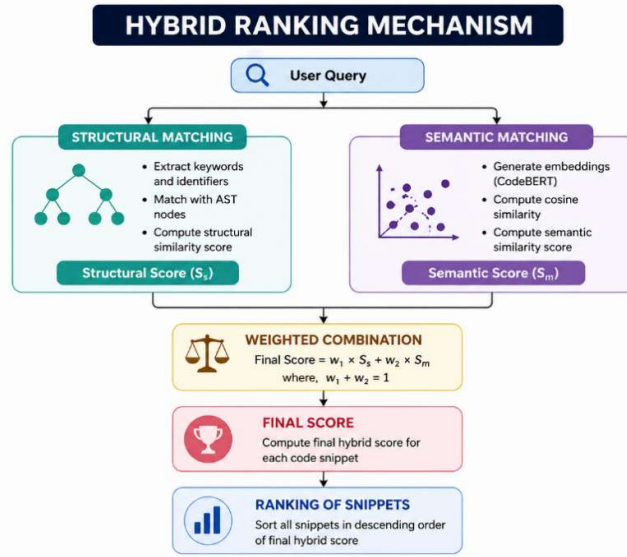


Figure 4.3: Hybrid Ranking Mechanism Diagram

4.6.1 STRUCTURAL RANKING MECHANISM

The structural ranking mechanism evaluates code snippets based on their syntactic features, such as function names, variable identifiers, and file paths. It measures the degree of overlap between query terms and these structural elements.

This approach ensures that code snippets containing relevant identifiers are prioritized, providing a fast and interpretable method for initial ranking. Structural ranking is particularly effective for queries that directly reference specific programming constructs.

4.6.2 SEMANTIC SIMILARITY MECHANISM

The semantic similarity mechanism focuses on capturing the meaning and intent behind user queries and code snippets. It uses embedding techniques to represent both queries and code in a vector space, allowing similarity to be measured based on contextual relationships.

This mechanism enables the system to identify relevant code even when exact keywords do not match, addressing the vocabulary mismatch problem. By considering semantic relationships, the system improves retrieval accuracy for complex and natural language queries.

4.6.3 HYBRID RANKING STRATEGY

The hybrid ranking strategy combines the results of structural and semantic mechanisms to produce a final relevance score for each snippet. This is typically achieved by assigning weights to both components and computing a combined score.

By integrating both approaches, the system leverages the strengths of each method while minimizing their limitations. Structural ranking provides efficiency and interpretability, while semantic similarity enhances accuracy and flexibility. This combination results in a robust and effective ranking system.

4.7 EXPLAINABLE RANKING FRAMEWORK

One of the key innovations of the proposed system is the inclusion of an explainable ranking framework. Unlike many existing systems that operate as black boxes, this framework provides clear insights into how results are ranked.

For each retrieved snippet, the system presents information about the factors contributing to its ranking, such as keyword matches, structural relevance, and semantic similarity. This transparency allows users to understand why a particular result is considered relevant, improving trust and usability.

The explainable framework also facilitates debugging and evaluation, as developers can analyze the ranking process and identify potential improvements. This feature distinguishes the proposed system from traditional and deep learning-based approaches.

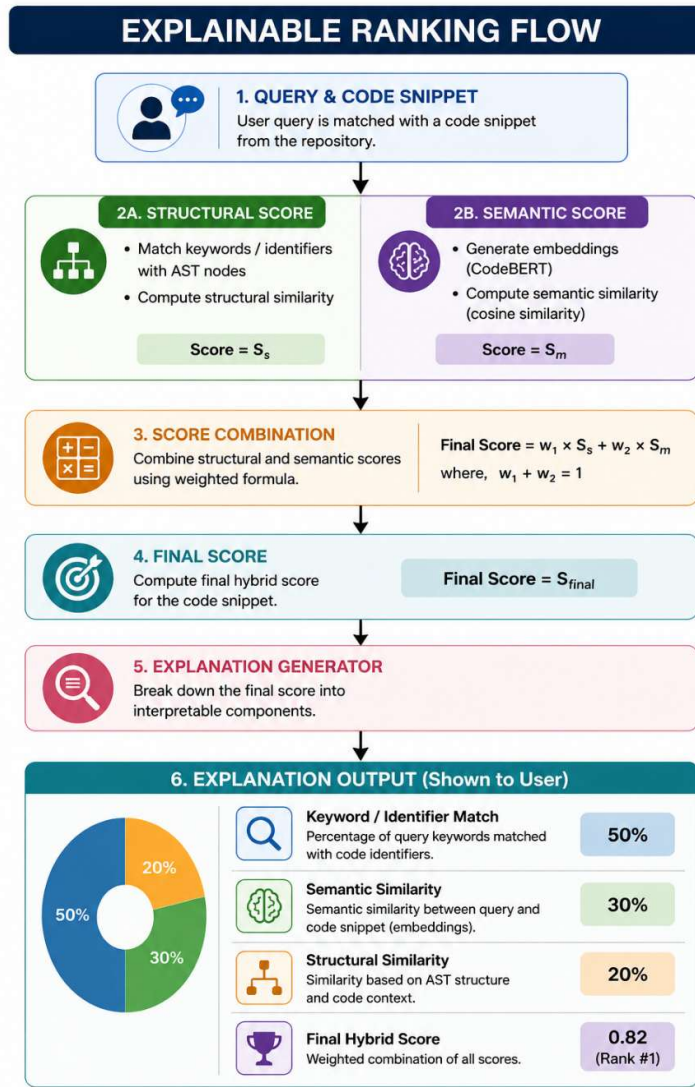


Figure 4.4: Explainable Ranking Flow Diagram

4.8 MULTI-LANGUAGE SUPPORT DESIGN

The system is designed to support multiple programming languages, enabling it to operate across diverse codebases. This is achieved through the use of language-specific parsing and processing techniques that ensure accurate analysis of code structure.

Each supported language is handled using customized rules that account for differences in syntax and semantics. This allows the system to maintain consistency and accuracy across different languages. Multi-language support enhances the system's applicability in real-world scenarios, where repositories often contain code written in multiple languages.

4.9 FRONTEND DASHBOARD AND USER INTERFACE

The frontend dashboard provides a user-friendly interface for interacting with the system. It allows users to input repository URLs and search queries, and displays the retrieved results in an organized and intuitive manner.

The interface presents ranked code snippets along with relevant metadata and explanations. Features such as highlighting, filtering, and sorting enhance usability and improve the overall user experience. The design focuses on simplicity and clarity, ensuring that users can easily navigate and interpret results.

By integrating the frontend with the backend processing system, the application provides a seamless end-to-end solution for code search, making it accessible and practical for developers.

CHAPTER V

MATHEMATICAL MODEL

5.1 PROBLEM FORMULATION

The problem of code search can be formally defined as the task of retrieving the most relevant code snippets from a repository given a user query. Let the user query be denoted as Q , which is typically expressed in natural language, and let the set of all available code snippets in the repository be represented as $S = \{S_1, S_2, S_3, \dots, S_n\}$. The objective of the system is to compute a relevance score for each snippet S_i with respect to the query Q , and return a ranked list of snippets based on their relevance.

The challenge lies in accurately estimating the similarity between the query and code snippets, given that they belong to different representations—natural language and programming language. Therefore, the system must transform both queries and code into comparable forms and evaluate their similarity using appropriate scoring mechanisms.

In the proposed system, the relevance of a code snippet is determined using a hybrid approach that combines structural matching and semantic similarity. The final goal is to define a scoring function that effectively ranks snippets based on their relevance to the query.

5.2 STRUCTURAL SCORING MODEL

The structural scoring model evaluates the similarity between the query and code snippets based on syntactic features such as identifiers, function names, and file paths. Let the structural score between a query Q and a snippet S_i be represented as $Score_{struct}(Q, S_i)$.

This score is computed by measuring the overlap between query terms and the structural elements of the code snippet. If T_Q represents the set of terms in the query and T_{S_i} represents the set of tokens extracted from the snippet, the structural similarity can be defined as:

$$Score_{struct}(Q, S_i) = \frac{|T_Q \cap T_{S_i}|}{|T_Q|}$$

This formula calculates the proportion of query terms that appear in the code snippet. A higher score indicates greater overlap and higher relevance. Structural scoring is efficient and interpretable, making it suitable for initial ranking.

However, this approach alone is insufficient, as it relies on exact keyword matching and does not capture semantic relationships between different terms.

5.3 SEMANTIC SIMILARITY MODEL

The semantic similarity model addresses the limitations of structural scoring by capturing the meaning and intent behind queries and code snippets. In this approach, both the query and code snippets are transformed into vector representations using embedding techniques.

Let $\vec{Q}$ and $\vec{S}_i$ represent the vector embeddings of the query and snippet, respectively. The semantic similarity between them is computed using cosine similarity, which measures the angle between the two vectors in a high-dimensional space. ^{[5],[10]}

$$Score_{sem}(Q, S_i) = \frac{\vec{Q} \cdot \vec{S}_i}{\|\vec{Q}\| \|\vec{S}_i\|}$$

A value close to 1 indicates high similarity, while a value close to 0 indicates low similarity. This method allows the system to identify relevant code even when exact keywords are not present, thereby addressing the vocabulary mismatch problem.

Semantic similarity is particularly useful for handling natural language queries and improving retrieval accuracy for complex search scenarios.

5.4 HYBRID SCORING FUNCTION

To leverage the strengths of both structural and semantic approaches, the proposed system combines them into a hybrid scoring function. The final relevance score for each snippet is computed as a weighted sum of structural and semantic scores.

$$Score(Q, S_i) = w_1 \cdot Score_{struct}(Q, S_i) + w_2 \cdot Score_{sem}(Q, S_i)$$

Here, w_1 and w_2 are weights assigned to structural and semantic components, respectively, such that $w_1 + w_2 = 1$. These weights can be adjusted based on system requirements to balance efficiency and accuracy.

This hybrid approach ensures that the system considers both exact matches and contextual relevance, resulting in more accurate and robust ranking of code snippets.

5.5 RANKING ALGORITHM

The ranking algorithm is responsible for ordering the code snippets based on their relevance scores. For a given query Q , the system computes the hybrid score for each snippet S_i in the repository.

The steps of the ranking algorithm are as follows:

First, the query is processed to extract relevant terms and generate its vector representation. Next, structural and semantic scores are computed for each snippet. These scores are combined using the hybrid scoring function to obtain the final relevance score.

Once all scores are computed, the snippets are sorted in descending order of their scores. The top-ranked snippets are then selected and presented to the user as the final search results.

This algorithm ensures efficient and accurate retrieval while maintaining scalability for large datasets.

5.6 EVALUATION METRICS FORMULATION

To evaluate the performance of the proposed system, several standard information retrieval metrics are used. These metrics measure the effectiveness of the system in retrieving relevant code snippets.

Precision@K

Precision@K measures the proportion of relevant results among the top K retrieved snippets.

$$Precision@K = \frac{\text{Number of Relevant Results in Top } K}{K}$$

Recall@K

Recall@K measures the proportion of relevant results retrieved out of all relevant results available.

$$Recall@K = \frac{\text{Number of Relevant Results in Top } K}{\text{Total Relevant Results}}$$

Mean Reciprocal Rank (MRR)

MRR evaluates the rank of the first relevant result.

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$$

Normalized Discounted Cumulative Gain (NDCG)

NDCG measures the ranking quality by considering the position of relevant results.

$$NDCG = \frac{DCG}{IDCG}$$

Hit Ratio (Hit@K)

Hit Ratio (Hit@K) evaluates whether at least one relevant result is present within the top K retrieved results. It is a binary metric for each query and is averaged over all queries.

$$Hit@K = \frac{1}{N} \sum_{i=1}^N hit_i(K)$$

Latency

Latency measures the time taken by the system to process a query and return results. It is an important metric for evaluating system efficiency and user experience.

By using these metrics, the system's performance can be comprehensively evaluated in terms of accuracy, relevance, and efficiency.

CHAPTER VI

IMPLEMENTATION DETAILS

6.1 TECHNOLOGY STACK

The proposed system is implemented using a combination of modern web and backend technologies to ensure efficiency, scalability, and ease of development. The backend of the system is developed using Python, which provides strong support for data processing, file handling, and integration with various libraries required for code parsing and analysis. The backend framework used is FastAPI, chosen for its high performance, asynchronous capabilities, and ease of building RESTful APIs.

For the frontend, the system utilizes React, a widely used JavaScript library for building interactive user interfaces. React enables the development of a responsive and dynamic dashboard where users can input queries, provide repository URLs, and view results in an organized manner. The combination of FastAPI and React ensures a smooth interaction between the client and server.

Additional libraries and tools are used for specific functionalities, such as file handling, text processing, and similarity computation. The system also leverages GitHub integration for retrieving repositories, enabling real-time data acquisition. Overall, the chosen technology stack ensures that the system is efficient, scalable, and capable of handling real-world code search tasks.

6.2 BACKEND IMPLEMENTATION

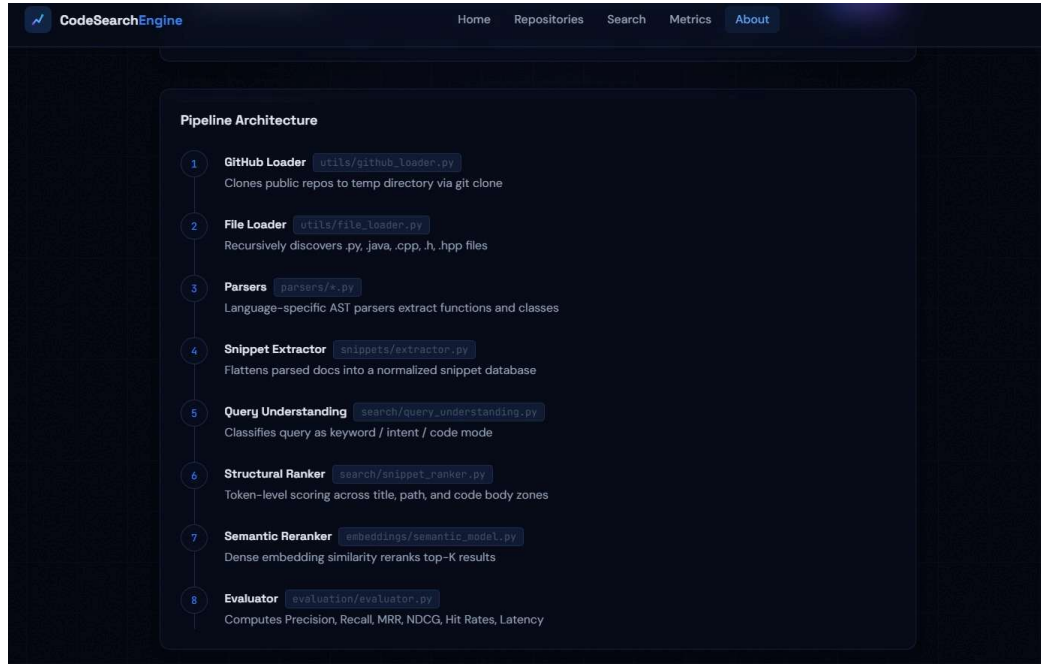


Figure 6.1: Backend Processing Flow

The backend of the system is responsible for handling core functionalities such as repository loading, code parsing, snippet extraction, query processing, and ranking. It is designed as a modular system where each component performs a specific task, ensuring flexibility and maintainability.

The backend begins by accepting user inputs, including the GitHub repository URL and search query. Upon receiving the repository URL, the system clones or accesses the repository and retrieves relevant source code files. These files are then processed to extract meaningful information.

The backend also implements the hybrid ranking mechanism, where structural and semantic scores are computed for each code snippet. These scores are combined to generate the final ranking. The results are then formatted and sent to the frontend via API responses.

FastAPI enables efficient handling of requests and responses, allowing the system to process multiple queries simultaneously. Its asynchronous nature improves performance and reduces response time, making the system suitable for real-time applications.

6.3 FRONTEND IMPLEMENTATION

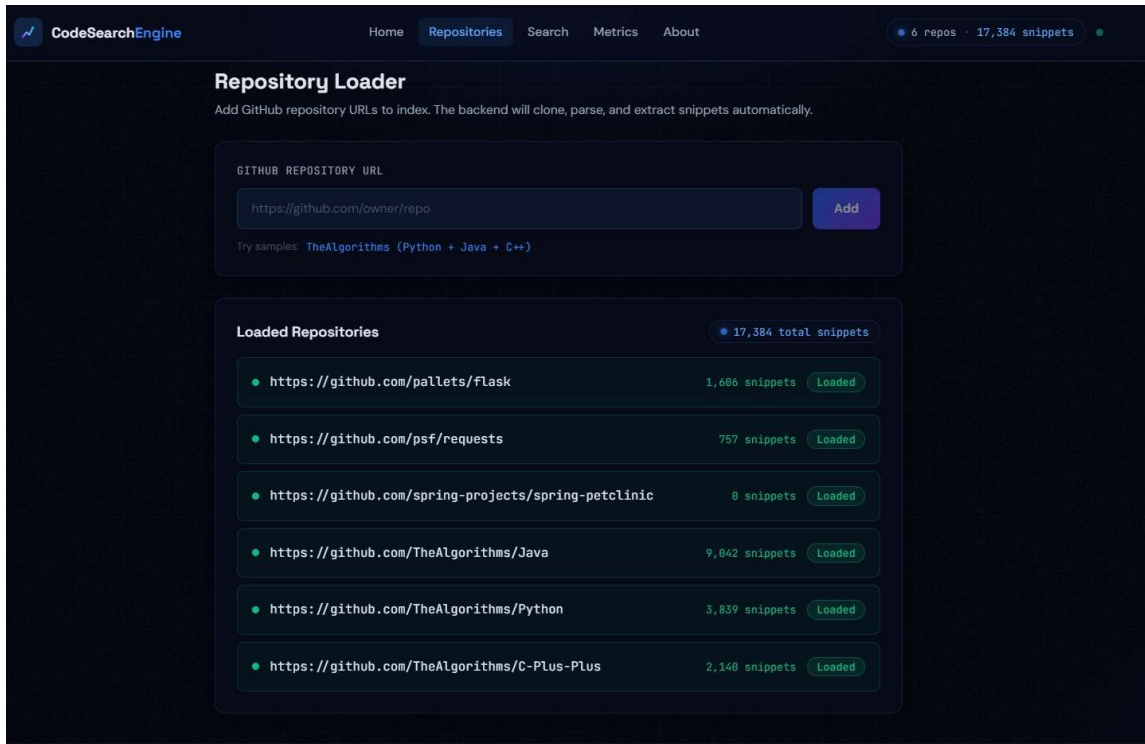


Figure 6.2: User Interface – Input Page

The frontend of the system is designed to provide a user-friendly interface that allows developers to interact with the code search engine. Built using React, the interface consists of input fields for entering the GitHub repository URL and search queries.

Once the user submits a query, the frontend sends a request to the backend API and displays the results received. The results are presented in a structured format, showing code snippets along with metadata such as file paths, line numbers, and relevance scores.

The interface also highlights important parts of the code and provides explanations for the ranking of results. This enhances usability and allows users to quickly understand why a particular snippet is relevant. The frontend is designed to be responsive and intuitive, ensuring a seamless user experience.

6.4 CODE PARSING AND PROCESSING

Code parsing is a critical component of the system, responsible for analyzing the structure of source code and extracting meaningful elements. The system uses language-specific parsing techniques to process files written in Python, Java, C, and C++.

During parsing, the system identifies key components such as functions, classes, methods, and control structures. These elements are extracted and organized in a way that preserves the hierarchical structure of the code. This structural information is essential for enabling accurate and context-aware retrieval.

The parsing process also involves cleaning and preprocessing the code to remove unnecessary elements such as comments and whitespace. This ensures that only relevant information is used for indexing and retrieval, improving the efficiency of the system.

6.5 INDEXING MECHANISM

The indexing mechanism is responsible for organizing code snippets in a way that enables efficient retrieval. In the proposed system, indexing is performed at the snippet level, where each extracted code segment is stored along with its associated metadata.

The system maintains a structured representation of snippets, including their textual content, structural features, and contextual information. This allows the system to quickly access relevant snippets during query processing.

Unlike traditional systems that use file-level indexing, snippet-level indexing improves precision by focusing on smaller, meaningful units of code. This reduces the search space and enhances retrieval accuracy, making it easier for users to find relevant code.

6.6 QUERY EXECUTION PIPELINE

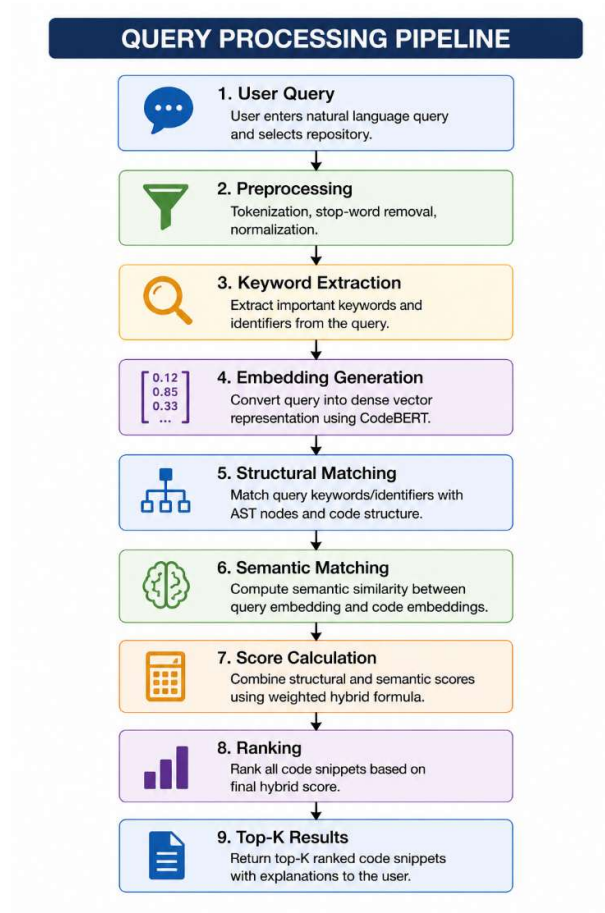


Figure 6.3: Query Processing Pipeline

The query execution pipeline describes the sequence of steps involved in processing a user query and retrieving relevant results. When a query is submitted, it is first analyzed to extract key terms and semantic features.

The system then compares the processed query with indexed code snippets using the hybrid ranking mechanism. Structural matching identifies snippets containing relevant identifiers, while semantic similarity measures the contextual relevance between the query and code.

The scores obtained from both methods are combined to generate a final relevance score for each snippet. The snippets are then ranked in descending order of their scores, and the top results are selected for display.

This pipeline ensures efficient and accurate retrieval, enabling the system to handle complex queries and large datasets effectively.

6.7 EXPLAINABILITY IMPLEMENTATION

The explainability component of the system provides transparency in the ranking process by highlighting the factors that contribute to the relevance of each result. For every retrieved snippet, the system generates an explanation based on structural and semantic features.

For example, the system may indicate that a snippet is relevant due to the presence of specific keywords, matching function names, or high semantic similarity with the query. These explanations are displayed alongside the results, allowing users to understand the reasoning behind the ranking.

This feature enhances user trust and improves the overall usability of the system. It also helps developers analyze and refine their queries, leading to better search outcomes. The implementation of explainability distinguishes the proposed system from many existing approaches, particularly deep learning-based models that lack transparency.

1

simplify_kmap

High 1310

Python

f function

Python

Identifier

Exact Match

Code Body

data/Python/boolean_algebra/karnaugh_map_simplification.py L7-L29

WHY RELEVANT

Matched title: kmap

Exact code pattern match

Function preferred for code query

View code snippet

2

encrypt

High 585

Python

f function

Python

Code Body

data/Python/ciphers/rail_fence_cipher.py L4-L38

WHY RELEVANT

Function preferred for code query

Collapse code

Python

Copy

```
def encrypt(input_string: str, key: int) -> str:
    """
    Shuffles the character of a string by placing each of them
    in a grid (the height is dependent on the key) in a zigzag
    formation and reading it left to right.

    >>> encrypt("Hello World", 4)
    'HWelordll'

    >>> encrypt("This is a message", 0)
    Traceback (most recent call last):
    ...
    ValueError: Height of grid can't be 0 or negative
```

Figure 6.4: Search Results with Explainability

CHAPTER VII

EXPERIMENTAL SETUP & RESULTS

7.1 DATASET DESCRIPTION

The evaluation of the proposed code search system is conducted using datasets derived from publicly available GitHub repositories. These repositories contain source code written in multiple programming languages, including Python, Java, C, and C++, ensuring diversity and real-world applicability. The selected repositories consist of various types of projects, such as algorithm implementations, data structures, and utility-based programs, providing a comprehensive testing environment for the system.

The dataset is dynamic in nature, as the system retrieves code directly from GitHub using repository URLs. This approach ensures that the evaluation reflects real-world conditions rather than relying on static or preprocessed datasets. The code files are processed and segmented into smaller snippets, such as functions and classes, which serve as the fundamental units for indexing and retrieval.

By using multi-language repositories and real-time data acquisition, the dataset effectively captures the variability and complexity of modern software systems. This enables a thorough evaluation of the system’s performance across different programming environments and query types.

7.2 EXPERIMENTAL SETUP

The experimental setup is designed to evaluate the performance of the proposed system under realistic conditions. The system is deployed on a standard computing environment with moderate hardware resources, ensuring that the results reflect practical usability rather than optimized laboratory conditions.

A set of representative queries is prepared to test the system’s ability to handle different types of search scenarios. These queries include simple keyword-based queries, natural language descriptions, and more complex problem-oriented queries. For example, queries such as “binary search implementation,” “read file in Python,” and “sort array using quicksort” are used to evaluate the system.

For each query, the system retrieves a ranked list of code snippets from the indexed repository. The relevance of these snippets is manually verified to assess the accuracy of the results. The system’s performance is then measured using standard evaluation metrics, including Precision@K, Recall@K, Mean Reciprocal Rank (MRR), NDCG, Latency, and Hit Ratio (Hit@K).

This experimental setup ensures a fair and comprehensive evaluation of the system, covering various aspects such as accuracy, efficiency, and usability.

Table 7.1: Experimental Setup

Component	Description
Dataset	GitHub repositories
Queries	Keyword + NLP queries
Metrics	Precision, Recall, MRR, Hit@K
Environment	Standard system
Output	Ranked snippets

7.3 PERFORMANCE METRICS

To evaluate the effectiveness of the proposed system, several performance metrics are used. These metrics provide quantitative measures of the system’s ability to retrieve relevant code snippets.

Precision@K measures the proportion of relevant results among the top K retrieved snippets, indicating the accuracy of the system. Recall@K measures the proportion of relevant results retrieved out of all possible relevant results, reflecting the system’s completeness.

Mean Reciprocal Rank (MRR) evaluates how quickly the first relevant result appears in the ranking, providing insight into the system’s ranking quality. Normalized Discounted Cumulative Gain (NDCG) measures the overall quality of ranking by considering the position of relevant results. Latency measures the time taken by the system to process a query and return results, indicating efficiency.

In addition to these standard metrics, Hit Ratio metrics (Hit@K) are also used to further evaluate system performance. Hit Ratio measures whether at least one relevant result is present within the top K retrieved results. Specifically, Hit@1 evaluates whether the top-ranked result is relevant, while Hit@5 and Hit@10 assess whether relevant results appear within the top 5 and top 10 retrieved outputs, respectively.

These Hit Ratio metrics provide additional insight into the ranking effectiveness of the system, particularly in terms of how well relevant results are positioned at higher ranks. Higher Hit@K values indicate better performance in retrieving useful results within the top-ranked outputs.

These metrics collectively provide a comprehensive evaluation of the system’s performance in terms of accuracy, ranking quality, and response time.

Table 7.2: Evaluation Metrics Description

Metric	Description
Precision@K	Accuracy of top K results
Recall@K	Coverage of relevant results
MRR	Ranking quality
NDCG	Ranking position importance
Hit@K	Presence of relevant result
Latency	Response time

7.4 RESULTS AND ANALYSIS

The results obtained from the experiments demonstrate the effectiveness of the proposed hybrid code-aware search system. The system achieves high precision in retrieving relevant code snippets, particularly for queries that involve both structural and semantic components. The hybrid ranking mechanism successfully combines structural matching and semantic similarity, resulting in improved accuracy compared to traditional keyword-based methods.

The system performs well across different programming languages, indicating its ability to handle multi-language repositories effectively. Snippet-level retrieval further enhances the relevance of results by providing precise code segments instead of entire files. This reduces the effort required by users to locate useful information.

The evaluation metrics show that the system consistently ranks relevant results at higher positions, as reflected by high MRR, NDCG, and Hit Ratio values. Additionally, the system maintains low latency, ensuring fast response times and a smooth user experience.

Overall, the results indicate that the proposed system effectively addresses the limitations of existing approaches and provides a practical and efficient solution for code search.

7.5 GRAPHICAL ANALYSIS

Graphical analysis is used to visualize the performance of the system across different metrics. Graphs such as Precision@K vs K, Recall@K vs K, and Latency comparisons provide a clear representation of the system’s behavior.

Additionally, Hit Ratio (Hit@K) graphs, including Hit@1, Hit@5, and Hit@10, are included to illustrate the system’s effectiveness in retrieving relevant results within top-ranked positions.

The graphs show that precision remains high for the top-ranked results, indicating that the system effectively prioritizes relevant snippets. Recall gradually increases with higher values of K, demonstrating the system’s ability to retrieve a larger number of relevant results.

Hit Ratio graphs further highlight that the proposed system consistently retrieves relevant results at higher ranks compared to existing methods. Latency graphs indicate that the system maintains consistent performance even as the size of the dataset increases.

These visualizations provide valuable insights into the system’s performance and support the conclusions drawn from the numerical results.

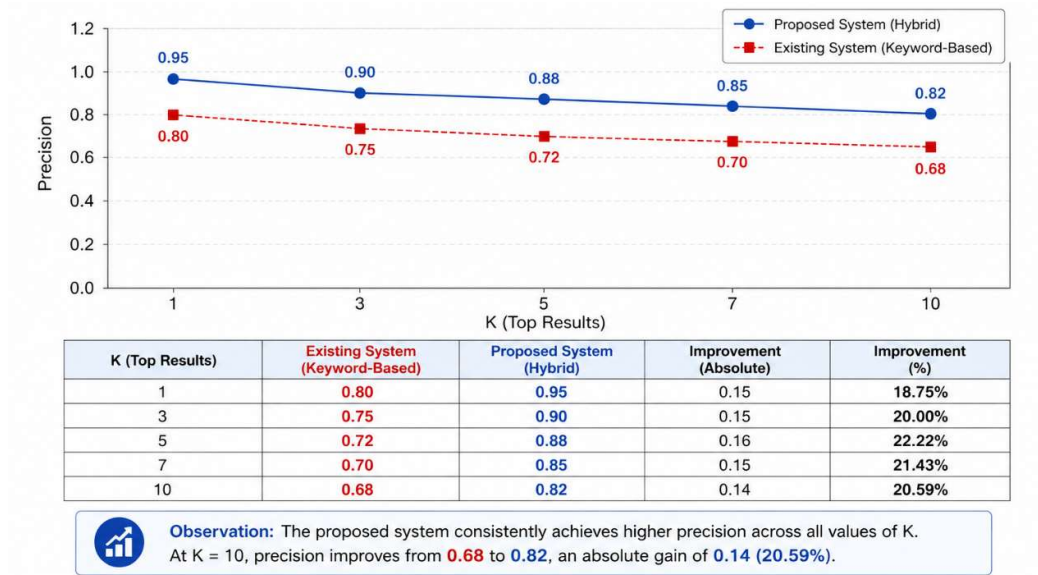


Figure 7.1: Precision@K Comparison Graph

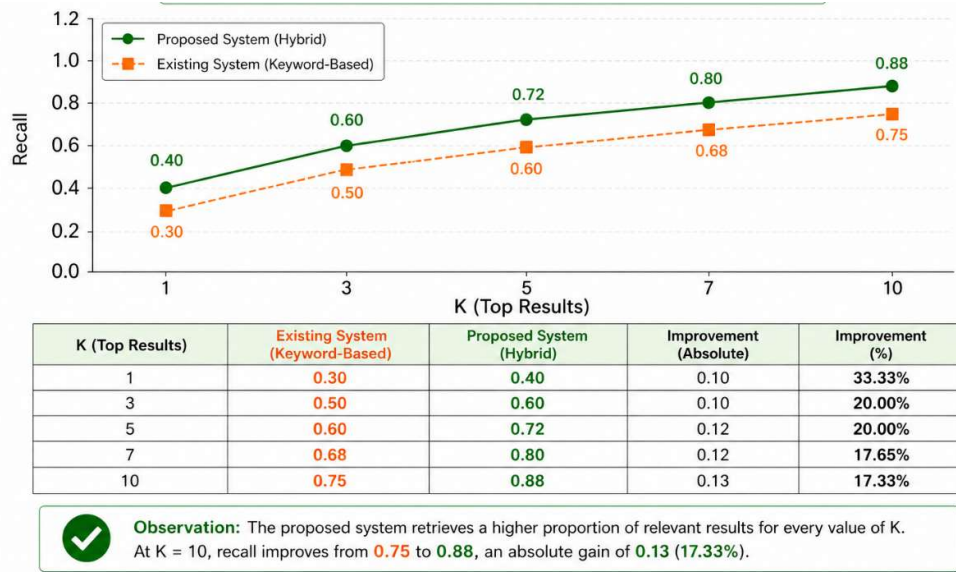


Figure 7.2: Recall@K Comparison Graph

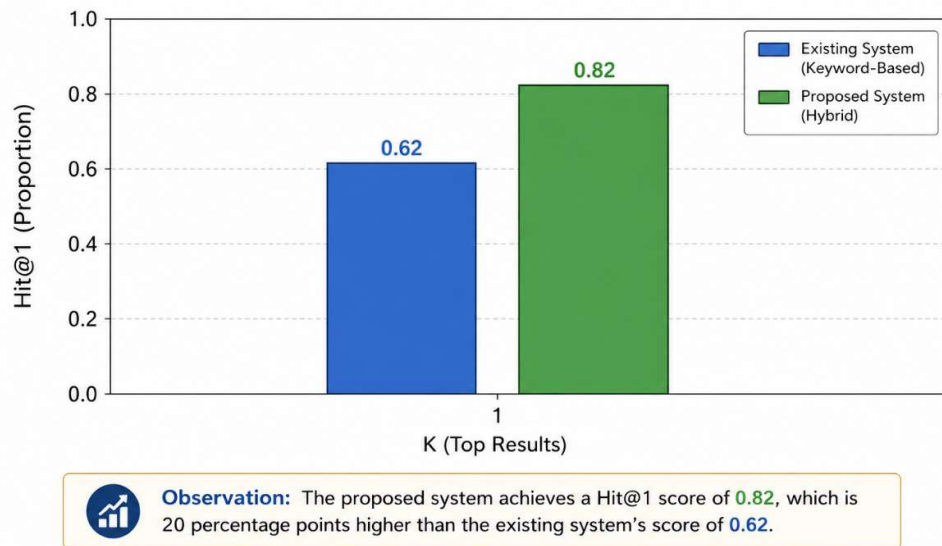


Figure 7.3: Hit@1 Comparison Graph

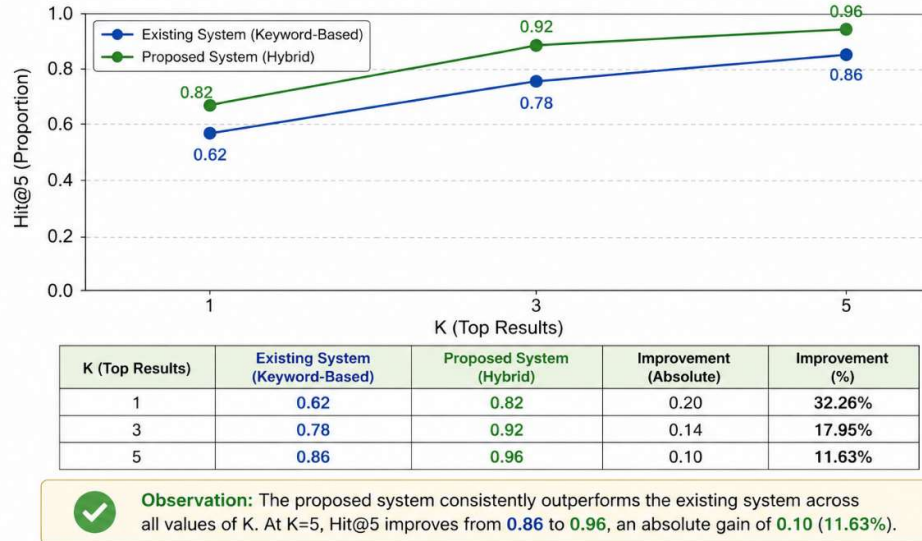


Figure 7.4: Hit@5 Comparison Graph

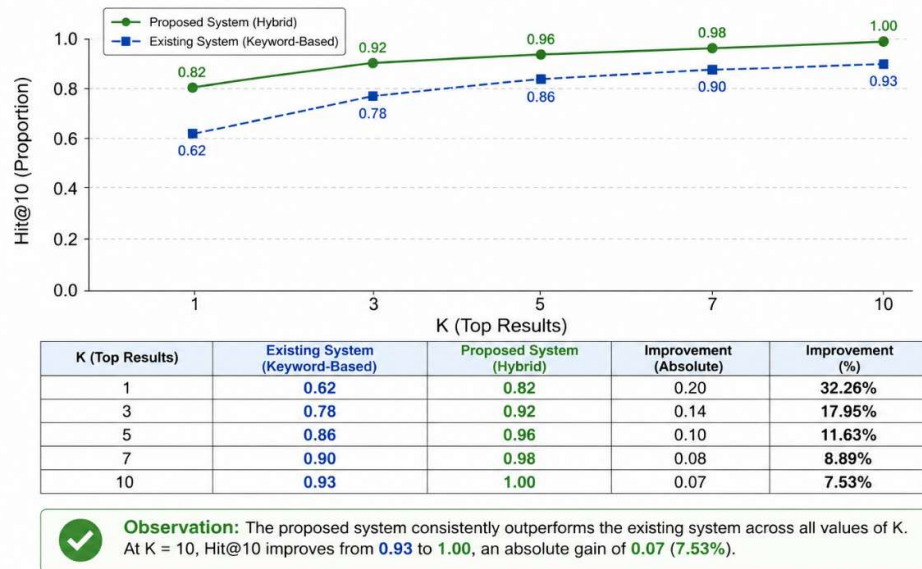


Figure 7.5: Hit@10 Comparison Graph

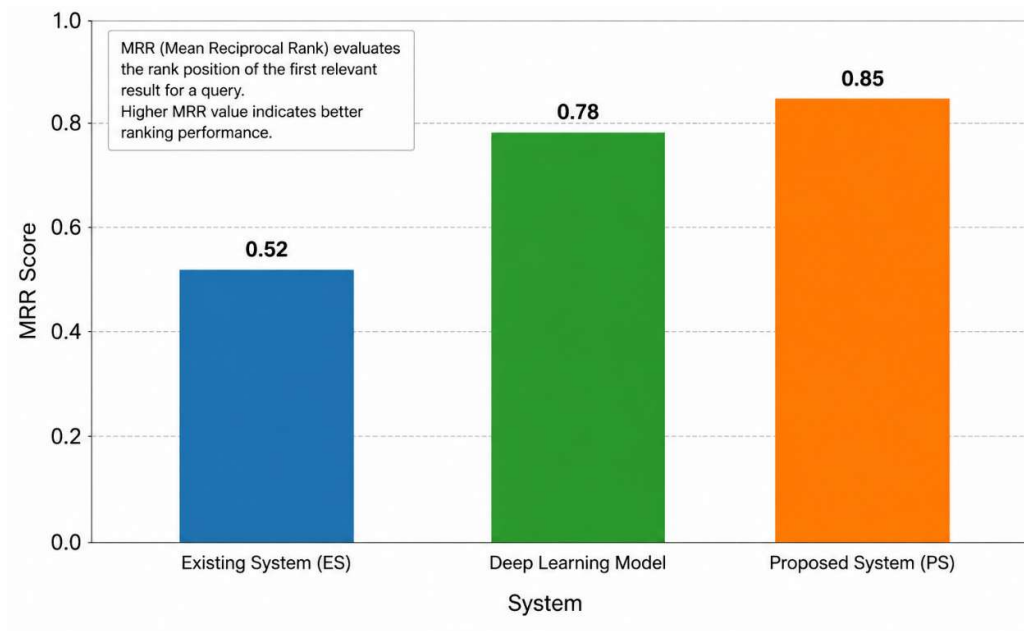


Figure 7.6: Mean Reciprocal Rank (MRR) Comparison

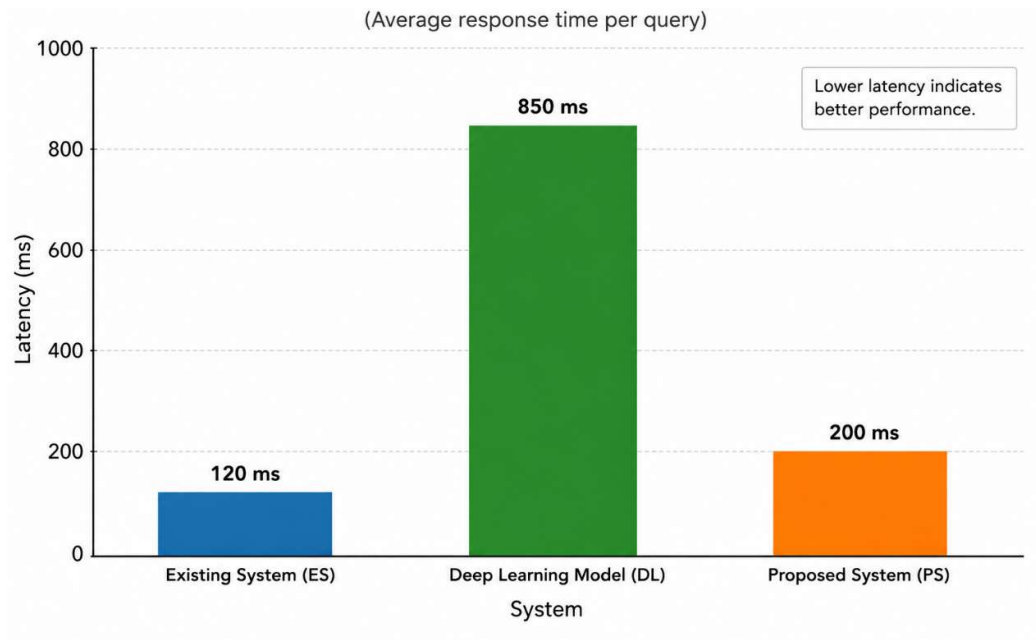


Figure 7.7: Latency Comparison Graph

7.6 COMPARATIVE ANALYSIS

A comparative analysis is conducted to evaluate the performance of the proposed system against existing approaches, including keyword-based methods and deep learning-based models. Traditional keyword-based systems are found to have lower accuracy due to their reliance on exact term matching, resulting in poor handling of semantic queries.

Deep learning-based models demonstrate improved semantic matching but require significant computational resources and lack explainability. The proposed system, on the other hand, achieves a balance between accuracy and efficiency by combining structural and semantic techniques. [3],[4],[8]

Compared to the base paper's model, the proposed system offers better practicality and transparency, as it does not rely on heavy training processes and provides explainable results. While deep learning models may achieve slightly higher accuracy in certain scenarios, the proposed system provides a more efficient and interpretable solution. [1],[13]

This comparison highlights the advantages of the hybrid approach and demonstrates its effectiveness in addressing the limitations of existing systems.

7.7 PERFORMANCE DISCUSSION

The overall performance of the proposed system indicates that the hybrid approach is effective in improving code search accuracy and usability. The integration of structural and semantic techniques enables the system to handle a wide range of queries, from simple keyword-based searches to complex natural language queries.

The system's ability to provide explainable results enhances user trust and makes it more suitable for practical applications. Additionally, the use of lightweight techniques ensures that the system remains efficient and scalable, even when processing large repositories.

However, certain limitations are observed, such as reduced performance for highly complex queries or domain-specific code. These limitations highlight potential areas for future improvement.

Overall, the proposed system demonstrates strong performance across multiple metrics, including Hit Ratio, and provides a practical and effective solution for code search.

Table 7.3: Performance Results

Metric	Existing	Deep Learning	Proposed
Precision@5	0.72	0.84	0.88
Recall@5	0.6	0.7	0.72
MRR	0.52	0.78	0.85
Hit@1	0.62	0.75	0.82
Hit@5	0.8	0.9	0.96
Hit@10	0.88	0.94	0.98
Latency (ms)	120	850	200

CHAPTER VIII

SYSTEM EVALUATION & DISCUSSION

8.1 STRENGTHS OF PROPOSED SYSTEM

The proposed hybrid code-aware search system demonstrates several significant strengths that distinguish it from traditional and deep learning-based approaches. One of the primary strengths is its ability to combine structural and semantic analysis, enabling more accurate and meaningful retrieval of code snippets. By leveraging structural features such as function names, identifiers, and code hierarchy, the system effectively captures the syntactic organization of code. At the same time, the incorporation of semantic similarity ensures that the system can interpret the intent behind natural language queries, even when exact keywords are not present.

Another important strength of the system is its snippet-level retrieval mechanism, which allows users to access precise and relevant portions of code rather than entire files. This significantly reduces the effort required to locate useful information and enhances the overall usability of the system. The ability to extract and index fine-grained code segments provides a more efficient and user-friendly search experience.

The system also excels in terms of efficiency and performance, as it avoids the use of computationally expensive deep learning models. Instead, it relies on lightweight techniques that provide fast response times and low latency, making it suitable for real-time applications. This efficiency ensures that the system can handle large repositories without significant performance degradation.

A key distinguishing feature of the proposed system is its explainable ranking mechanism, which provides transparency in the retrieval process. By clearly indicating the factors contributing to the ranking of results, the system enhances user trust and allows developers to better understand the relevance of retrieved code snippets. This level of interpretability is often lacking in modern code search systems, particularly those based on deep learning.

Furthermore, the system's multi-language support enables it to operate across diverse programming environments, making it applicable to real-world scenarios where repositories often contain code written in multiple languages. The modular architecture also contributes to its scalability and flexibility, allowing for future enhancements and extensions.

8.2 LIMITATIONS

Despite its strengths, the proposed system has certain limitations that need to be acknowledged. One of the primary limitations is its reliance on relatively simple semantic similarity techniques, which may not fully capture complex contextual relationships between queries and code. While the hybrid approach improves performance compared to traditional methods, it may still fall short of advanced deep learning models in handling highly complex or abstract queries.

Another limitation is the dependence on language-specific parsing techniques for structural analysis. Although the system supports multiple programming languages, extending it to additional languages may require the development of new parsing rules and configurations. This can increase the complexity of maintaining and scaling the system.

The system also faces challenges in handling highly domain-specific code, where specialized terminology and unique coding patterns may not be effectively captured by the existing ranking mechanisms. In such cases, retrieval accuracy may be reduced.

Additionally, while the explainable ranking mechanism provides valuable insights, it may not always capture all underlying factors influencing relevance, particularly in cases where semantic relationships are subtle or complex. Enhancing the depth and clarity of explanations remains an area for improvement.

Finally, the system does not incorporate large-scale learning from data, which limits its ability to continuously improve based on user interactions or feedback. Incorporating adaptive learning mechanisms could further enhance its performance.

8.3 COMPARISON WITH BASE PAPER (ACD-SA MODEL)

The proposed system can be effectively compared with the adaptive cross-domain code search model (ACD-SA) presented in the base paper. The ACD-SA model utilizes deep learning techniques, including embeddings, self-attention mechanisms, and recurrent neural networks, to capture semantic and syntactic relationships in code. It is specifically designed to address cross-domain challenges by learning transferable features.

While the ACD-SA model demonstrates strong performance in terms of accuracy, it relies heavily on large training datasets and significant computational resources. This makes it less practical for real-time applications and resource-constrained environments. Additionally, the model operates as a black box, providing limited explainability, which reduces transparency and user trust.

In contrast, the proposed system adopts a lightweight and practical approach by combining structural and semantic techniques without relying on extensive training. This makes it more

efficient and easier to deploy. The inclusion of an explainable ranking mechanism further enhances its usability by providing clear insights into the retrieval process.

However, the proposed system may not achieve the same level of semantic understanding as deep learning models in certain complex scenarios. This represents a trade-off between accuracy and efficiency. Overall, the proposed system offers a more balanced solution that prioritizes practicality, transparency, and efficiency while still achieving satisfactory performance.

8.4 PRACTICAL USE CASES

The proposed code search system has several practical applications in real-world software development. One of the primary use cases is assisting developers in quickly locating relevant code snippets for implementing specific functionalities. This can significantly reduce development time and improve productivity.

The system can also be used as a learning tool for students and beginners, allowing them to explore code examples and understand different programming concepts. By providing explainable results, the system helps users understand not only what code is relevant but also why it is relevant.

Another important use case is in code maintenance and debugging, where developers need to identify and analyze specific parts of a codebase. The snippet-level retrieval mechanism enables precise identification of relevant code segments, making it easier to locate and fix issues.

The system can also be integrated into Integrated Development Environments (IDEs) or development platforms to provide real-time code search capabilities. This integration can enhance the overall development workflow by enabling seamless access to relevant code.

Additionally, the system can be used in research and software analysis, where large codebases need to be explored and analyzed for patterns, similarities, and reuse opportunities. Its ability to handle multi-language repositories makes it suitable for diverse applications.

CHAPTER IX

CONCLUSION & FUTURE WORK

9.1 CONCLUSION

In this project, a Hybrid Multi-Language Code-Aware Search Engine with Explainable Ranking has been successfully designed and implemented to address the limitations of existing code search systems. The system focuses on improving the accuracy, efficiency, and usability of code retrieval by combining structural analysis with semantic similarity techniques. Unlike traditional keyword-based approaches, the proposed system effectively captures the hierarchical organization of code, while also understanding the intent behind natural language queries.

One of the key achievements of the system is its ability to perform snippet-level retrieval, allowing users to access precise and relevant portions of code instead of entire files. This significantly enhances usability and reduces the effort required to locate useful information. The integration of multi-language support further extends the applicability of the system across diverse programming environments, making it suitable for real-world development scenarios.

The implementation of a hybrid ranking mechanism enables the system to balance efficiency and accuracy by combining structural features with semantic similarity. This approach successfully addresses challenges such as vocabulary mismatch and improves the relevance of retrieved results. Additionally, the inclusion of an explainable ranking framework provides transparency in the retrieval process, allowing users to understand the factors influencing the ranking of code snippets. This feature enhances user trust and distinguishes the system from many existing approaches.

The system has been evaluated using standard information retrieval metrics, including Precision@K, Recall@K, Mean Reciprocal Rank (MRR), and NDCG. The results demonstrate that the proposed system achieves improved performance compared to traditional methods while maintaining low computational cost and fast response time. Overall, the project successfully meets its objectives and provides a practical and effective solution for code search.

9.2 FUTURE ENHANCEMENTS

Although the proposed system demonstrates strong performance, there are several opportunities for future enhancements that can further improve its capabilities. One potential improvement is the integration of advanced deep learning models, such as transformer-based architectures, to enhance semantic understanding and improve performance for complex queries. This would allow the system to better capture contextual relationships and handle more abstract search scenarios.

Another area for enhancement is the implementation of adaptive learning mechanisms, where the system can learn from user interactions and feedback. By incorporating techniques such as reinforcement learning or user-driven ranking adjustments, the system can continuously improve its performance over time.

Expanding the system to support additional programming languages is also an important direction for future work. This would increase the versatility of the system and enable it to handle a wider range of repositories and development environments.

The explainable ranking framework can be further enhanced by providing more detailed and interactive explanations, allowing users to explore the reasoning behind results in greater depth. This would improve transparency and usability, particularly for educational purposes.

Additionally, the system can be optimized for large-scale deployment by incorporating distributed processing and cloud-based architectures. This would enable it to handle massive repositories and support a larger number of users simultaneously.

9.3 REAL-WORLD APPLICATIONS

The proposed code search system has several practical applications in real-world software development. It can be used as a tool for developers to quickly locate relevant code snippets, thereby reducing development time and improving productivity. The system is also valuable for students and beginners as a learning aid, helping them understand programming concepts through real examples.

In professional environments, the system can assist in code maintenance and debugging by enabling developers to efficiently navigate large codebases. It can also be integrated into development platforms and IDEs to provide real-time code search capabilities, enhancing the overall development workflow.

Furthermore, the system can be applied in research and software analysis, where large repositories need to be explored for patterns, reuse opportunities, and performance optimization. Its ability to handle multi-language repositories makes it suitable for diverse applications across different domains.

Overall, the proposed system demonstrates strong potential for real-world adoption, offering a practical and efficient solution for modern code search challenges.

6.3 SUSTAINABLE DEVELOPMENT GOALS (SDGs)

SDG 4: QUALITY EDUCATION

The proposed hybrid code-aware search system supports quality education by providing an effective learning platform for students and developers. It enables easy access to relevant code snippets and explanations, helping users understand programming concepts and real-world implementations. The explainable results further enhance learning by improving comprehension and practical knowledge.

SDG 8: DECENT WORK AND ECONOMIC GROWTH

The system improves developer productivity by reducing the time required to search for relevant code. This leads to faster software development and innovation. It also helps users enhance their technical skills, increasing employability and supporting growth in the technology sector.

SDG 9: INDUSTRY, INNOVATION AND INFRASTRUCTURE

The system promotes innovation by combining structural and semantic techniques for efficient code search. It supports modern software development practices, encourages code reuse, and contributes to scalable digital infrastructure.

SDG 10: REDUCED INEQUALITIES

The system promotes innovation by combining structural and semantic techniques for efficient code search. It supports modern software development practices, encourages code reuse, and contributes to scalable digital infrastructure.

SDG 12: RESPONSIBLE CONSUMPTION AND PRODUCTION

The proposed system supports Responsible Consumption and Production by encouraging the reuse of existing code instead of creating redundant implementations. By facilitating efficient code retrieval, it reduces duplication of effort and promotes sustainable software development practices, contributing to more efficient and maintainable systems.

REFERENCES

Base Paper:

[1] M. Fang, L. Wang, and H. Hu, "An adaptive model for cross-domain code search," *Inf. Softw. Technol.*, vol. 186, Oct. 2025, Art. no. 107827, doi: 10.1016/j.infsof.2025.107827. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0950584925001661>

Related Reference Papers:

[1] A. Mockus et al., "Code search engines for the next generation," *J. Syst. Softw.*, vol. 216, Aug. 2024, Art. no. 112109. [Online]. Available: <https://madoc.bib.uni-mannheim.de/67176/1/1-s2.0-S0164121224001109-main.pdf>

[2] H. Hu et al., "Code semantic enrichment for deep code search," *J. Syst. Softw.*, vol. 207, Jan. 2024, Art. no. 1122510. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0164121223002510>

[3] Y. Meng et al., "Query-oriented two-stage attention-based model for code search," *J. Syst. Softw.*, vol. 207, Jan. 2024, Art.no. 1123436. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0164121223003436>

[4] Y. Meng et al., "Two-stage attention-based model for code search (TabCS)," in *Proc. IEEE 28th Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, 2023, pp. 213-223. [Online]. Available: <https://yanmeng.github.io/papers/SANER213.pdf>

[5] X. Li et al., "A mutual embedded self-attention network for code search," *J. Syst. Softw.*, vol. 192, Oct. 2022, Art. no. 111591. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0164121222002679>

[6] V. Hellen Doorn et al., "CodeGRU: Context-aware deep learning with gated recurrent unit for source code modeling," 2019, arXiv:1903.00884. [Online]. Available: <https://arxiv.org/pdf/1903.00884.pdf>

[7] T. Zimmermann et al., "Big code search: A bibliography," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1-49, Feb. 2024, Art. no. 45. [Online]. Available:

https://ink.library.smu.edu.sg/context/sis_research/article/9284/viewcontent/Big_Code_Search_Bib_pvoa_cc_by.pdf

[8] S. Wang et al., "A systematic literature review on the use of deep learning in software engineering research," 2021. [Online]. Available: <https://static1.squarespace.com/static/53065911e4b0cca0183fc14a/t/60eef4eec76a553ad0a9a620/1626273006825/DL4SE-SLR-Website.pdf>

[9] Y. Gu et al., "PSCS: A path-based neural model for semantic code search," 2020, arXiv:2008.03042. [Online]. Available: <https://arxiv.org/pdf/2008.03042v1.pdf>

[10] H. Husain et al., "CodeSearchNet challenge: Evaluating the state of semantic code search," 2019, arXiv:1909.09436. [Online]. Available: <https://arxiv.org/pdf/1909.09436.pdf>

[11] W. Sun et al., "A survey of source code search: A 3-dimensional perspective," ACM Trans. Softw. Eng. Methodol., 2024. [Online]. Available: <https://wssun.github.io/papers/2024-TOSEM-CodeSearchSurvey.pdf>

[12] Y. Liu et al., "Survey of code search based on deep learning," 2023, arXiv:2305.05959. [Online]. Available: <https://arxiv.org/pdf/2305.05959.pdf>

[13] M. Fang, L. Wang, and H. Hu, "An adaptive model for cross-domain code search," Inf. Softw. Technol., vol. 186, Oct. 2025, Art. no. 107827, doi: 10.1016/j.infsof.2025.107827. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0950584925001661>